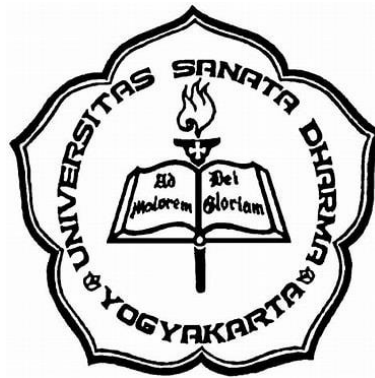


**IDENTIFIKASI URL SITUS BERBAHAYA MENGGUNAKAN
JARINGAN SARAF TIRUAN CNN-LSTM**

SKRIPSI

Diajukan Untuk Memenuhi Salah Satu Syarat
Memperoleh Gelar Sarjana Komputer
Program Studi Informatika



Disusun oleh:

Mathys Jorge Alberino Seilatu

NIM: 205314714

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS SANATA DHARMA
YOGYAKARTA**

2024

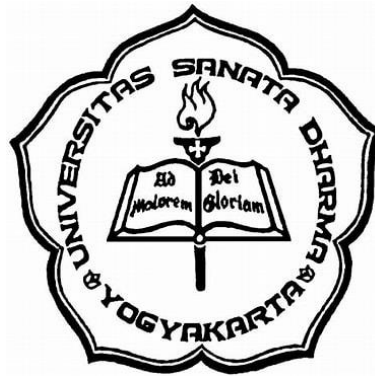
MALICIOUS URLS IDENTIFICATION USING NEURAL NETWORK CNN-LSTM

THESIS

Present as Partial Fullfillment of the Requirements

For The Degree of Sarjana Komputer

In Informatics Department



By:

Mathys Jorge Alberino Seilatu

NIM: 205314714

**INFORMATICS STUDY PROGRAM
DEPARTMENT OF INFORMATICS
FACULTY OF SCIENCE AND TECHNOLOGY
SANATA DHARMA UNIVERSITY
YOGYAKARTA**

2024

**HALAMAN PERSETUJUAN PEMBIMBING
SKRIPSI**

**IDENTIFIKASI URL SITUS BERBAHAYA MENGGUNAKAN
JARINGAN SARAF TIRUAN CNN-LSTM**

Disusun oleh:

Mathys Jorge Alberino Seilatu

NIM: 205314714

Telah disetujui oleh :

Dosen Pembimbing,

Cyprianus Kuntoro Adi, S.J., M.A., M.Sc., Ph.D.

Mei 2024

**HALAMAN PENGESAHAN
SKRIPSI**

**IDENTIFIKASI URL SITUS BERBAHAYA MENGGUNAKAN
JARINGAN SARAF TIRUAN CNN-LSTM**

Dipersiapkan oleh dan ditulis oleh :

Mathys Jorge Alberino Seilatu

NIM: 205314714

SUSUNAN DEWAN PENGUJI

HALAMAN KEASLIAN KARYA

Saya menyatakan dengan sesungguhnya bahwa skripsi yang saya tulis tidak mengandung atau memuat karya orang lain, kecuali yang telah disebutkan dalam daftar pustaka dengan mengikuti ketentuan sebagaimana layaknya karya ilmiah.

Apabila di kemudian hari ditemukan indikasi plagiarisme dalam naskah ini, saya bersedia menanggung segala sanksi sesuai peraturan perundang-undangan yang berlaku.

Yogyakarta, ... Mei 2024

Penulis

Mathys Jorge Alberino Seilatu

HALAMAN PERSETUJUAN PUBLIKASI

Yang bertanda tangan di bawah ini, saya mahasiswa Universitas Sanata Dharma :

Nama : Mathys Jorge Alberino Seilatu

NIM : 205314174

Demi pengembangan ilmu pengetahuan, saya memberikan kepada Perpustakaan Universitas Sanata Dharma karya ilmiah saya yang berjudul :

“IDENTIFIKASI URL SITUS BERBAHAYA MENGGUNAKAN JARINGAN SARAF TIRUAN CNN-LSTM”

Beserta perangkat yang diperlukan (bila ada). Dengan demikian saya memberikan hak kepada Perpustakaan Universitas Sanata Dharma baik untuk menyimpan, mengalihkan dalam bentuk media lain, mengolah dalam bentuk pangkalan data, mendistribusikan secara terbatas, dan mempublikasikannya di internet atau media lain untuk kepentingan akademis tanpa perlu meminta izin dari saya atau memberikan royalti kepada saya selama tetap mencantumkan nama saya sebagai penulis.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di Yogyakarta

Pada tanggal ... Mei 2024

Yang menyatakan,

Mathys Jorge Alberino Seilatu

HALAMAN MOTTO

KATA PENGANTAR

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR.....	ix
DAFTAR TABEL	x
DAFTAR LAMPIRAN	xi
ABSTRAK	xii
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Rumusan Masalah.....	2
1.3. Batasan Masalah	2
1.4. Tujuan Penelitian	3
1.5. Manfaat Penelitian	3
BAB II TINJAUAN PUSTAKA.....	4
2.1. Landasan Teori	4
2.1.1. URL	4
2.1.2. Convolutional Neural Network	5
2.1.3. Word Embedding	8
2.1.4. Tokenizing	8
2.1.5. Long Short Term Memory (LSTM).....	9
2.1.6. K-Fold Cross Validation	11
2.2 Tinjauan Pustaka.....	12
BAB III METODE PENELITIAN	15
3.1. Gambaran Umum Penelitian	15
3.2. Deskripsi Data	16
3.3. Tahapan Penelitian	17
3.3.1. Eksploratory Data	17
3.3.2. Label Encoding.....	21
3.3.3. Cleaning dan Normalize	22
3.3.4. Tokenize	23
3.3.5. Layer Embedding	26
3.3.6. Layer Convolutional Neural Network	27
3.3.7. Optimalisasi Resource Hardware.....	29
3.3.8. Evaluasi	30
3.4. Skenario Pengujian	30

3.5.	Kebutuhan Perangkat Hardware dan Software	31
BAB IV HASIL PENELITIAN DAN PEMBAHASAN.....		33
4.1	Hasil Skenario Pengujian	33
4.2	Analisa Hasil Pengujian.....	34
4.2.1	Pengujian Tanpa K-Fold.....	34
4.2.2	Pengujian Dengan K-Fold	39
BAB V KESIMPULAN DAN SARAN		43
5.1.	Kesimpulan.....	43
5.2.	Saran.....	44
DAFTAR PUSTAKA.....		45
LAMPIRAN.....		47

DAFTAR GAMBAR

Gambar 2.1.1 Struktur Tautan (https://blog.hubspot.com/marketing/parts-url).....	4
Gambar 2.1.2 Contoh link phishing dengan membedakan font (https://www.bca.co.id/en/informasi/awas-modus/2023/06/26/07/22/waspada-website-palsu-menggunakan-abjad-karakter-yang-mirip-dengan-aslinya)	5
Gambar 2.1.3 Use-case tiap Neural Network (T. Oliver, 2020).....	7
Gambar 2.1.4 CNN 1D (Graeme Hirst, dkk, 2017)	7
Gambar 2.1.5 CNN 1D 2 (Graeme Hirst, dkk, 2017)	7
Gambar 2.1.6 Ilustrasi Word Embedding (https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4).....	8
Gambar 2.1.7 Tokenizing (Alexandra George, 2018).....	8
Gambar 2.1.8 Arsitektur LSTM (G. Aurélien, 2019).....	10
Gambar 3.1.1 Diagram alur penelitian.....	15
Gambar 3.2.1 Dataset whitelist dari GitHub.....	16
Gambar 3.2.2 Dataset dari Kaggle.....	17
Gambar 3.3.1 Persebaran Data.....	18
Gambar 3.3.2 data setelah balancing dan ditambah whitelist	18
Gambar 3.3.3 Wordcloud keseluruhan data	19
Gambar 3.3.4 Wordcloud data defacement	19
Gambar 3.3.5 Wordcloud data phishing.....	20
Gambar 3.3.6 Wordcloud data benign.....	20
Gambar 3.3.7 Wordcloud data malware.....	21
Gambar 3.3.8 Blok Kode untuk label encoding.....	22
Gambar 3.3.9 Blok Kode untuk cleaning.....	22
Gambar 3.3.10 Daftar punctuation dalam library String bawaan python	22
Gambar 3.3.11 Blok kode untuk memecah url.....	23
Gambar 3.3.12 Blok kode untuk memasukan url ke masing masing array.....	23
Gambar 3.3.13 Blok kode penggabungan kembali	24
Gambar 3.3.14 Hasil tokenisasi	25
Gambar 3.3.15 blok kode tokenisasi menggunakan library	26
Gambar 3.3.16 output neuron 2	27
Gambar 3.3.17 output neuron 4	27
Gambar 3.3.18 CNN neuron 4	28
Gambar 3.3.19 Blok kode optimalisasi	29
Gambar 3.3.20 Blok kode fungsi folding.....	30
Gambar 3.3.21 Blok kode perulangan dalam satu fold.....	30
Gambar 4.2.1 Grafik Akurasi Arsitektur 3LSTM 1CNN	34
Gambar 4.2.2 Grafik Akurasi Arsitektur 2LSTM 2CNN	35
Gambar 4.2.3 Grafik Akurasi Arsitektur 1LSTM 3CNN	35
Gambar 4.2.4 Grafik Akurasi Arsitektur 4CNN	36
Gambar 4.2.5 Grafik Akurasi Arsitektur 4LSTM	36
Gambar 4.2.6 Grafik Akurasi Arsitektur 2LSTM 1CNN	37
Gambar 4.2.7 Grafik Rata Rata Akurasi Learning Rate.....	37
Gambar 4.2.8 Grafik Heatmap Korelasi Tanpa K-Fold	38
Gambar 4.2.9 Akurasi Data Baru dengan K-Fold.....	39

Gambar 4.2.10 Akurasi Data Latih dengan K-Fold	39
Gambar 4.2.11 Pengujian Pertama Crash.....	39
<i>Gambar 4.2.12</i> Pengujian dengan Melakukan Fold Satu Persatu.....	40
Gambar 4.2.13 Grafik Heatmap Korelasi K-Fold.....	40
Gambar 4.2.14 Grafik Persebaran Puncak Akurasi Tertinggi	41

DAFTAR TABEL

Tabel 2.2.1 Tabel tinjauan pustaka	12
Tabel 3.3.1 Uji optimalisasi resource pertama	29
Tabel 3.3.2 Uji optimalisasi resource kedua	29
Tabel 3.4.1 Skenario Pengujian Dengan K-Fold dan Tanpa K-Fold.....	30
Tabel 4.1.1 Hasil Uji dengan K-Fold	33
Tabel 4.1.2 Hasil Uji tanpa K-Fold	33

DAFTAR LAMPIRAN

Lampiran 1 Gambar Contoh Hasil Layer LSTM	47
Lampiran 2 Seluruh Data Pengujian tanpa K-Fold	48
Lampiran 3 Seluruh Data Pengujian Menggunakan K-Fold.....	50
Lampiran 4 Arsitektur Model Terbaik Pengujian K-Fold	52
Lampiran 5 Grafik akurasi dan loss tiap iterasi 3-fold.....	53
Lampiran 6 Grafik akurasi dan loss tiap iterasi 5-fold.....	54
Lampiran 7 Grafik akurasi dan loss tiap iterasi 7-fold.....	55
Lampiran 8 Grafik akurasi dan loss tiap iterasi 9-fold.....	56
Lampiran 9 Error yang didapat tiap run.....	57

ABSTRAK

Tautan keamanan web makin bertambah setiap harinya, dan deteksi serta analisis halaman web yang berbahaya makin penting dan sulit dilakukan, *Phishing* adalah salah satu bentuk kejahatan dengan cara mendapatkan informasi sensitif dengan meniru pengirim yang terpercaya di dalam saluran komunikasi jika dalam dunia internet berarti meniru *domain name* dari sebuah *website* sehingga pengguna internet tertipu dan akhirnya memberikan data pribadi mereka.

Dikarenakan perkembangan teknologi dan membludaknya *URL* (tautan) di internet maka kita tidak dapat melakukan pengecekan satu persatu terhadap tautan tautan yang ada, maka dibangunlah sistem untuk membantu dalam hal tersebut.

Prediksi *URL* berbahaya dibangun menggunakan *Deep Learning* dengan metode campuran *Long Short term Memory* (LSTM) dan *Convolutional Neural Network* (CNN). Pelatihan dan pengujian model tersebut dilakukan pada platform *Kaggle Notebook*. Setelah selesai, model akan dievaluasi akurasi dan *f1* skornya.

Kata kunci : tautan berbahaya, *neural network*, *phishing*, *fraud detection*, *kaggle notebook*.

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dengan perkembangan teknologi internet yang terus berlangsung, tautan keamanan *web* makin bertambah setiap harinya, dan deteksi serta analisis halaman *web* yang berbahaya makin penting dan sulit dilakukan [1]. *Phishing* adalah salah satu bentuk kejahatan dengan cara mendapatkan informasi sensitif dengan meniru pengirim yang terpercaya di dalam saluran komunikasi. *Phishing* sering digunakan untuk mendapatkan informasi atau uang dengan cara yang tidak benar. Biasanya, pesan yang dikirimkan mengandung *software* atau tautan berbahaya. Lebih mudah untuk membuat laman palsu daripada membobol keamanan sistem. Selain itu, *phishing* dapat diluncurkan dengan biaya yang relatif murah dari mana saja di dunia karena internet yang terbuka dan anonim. Laman *phishing* berisi logo tipe dan teks yang ditujukan untuk meniru laman yang asli [2].

Dikarenakan maraknya hal tersebut menjadikan jenis *fraud* ini mulai berkembang dan diterapkan pada halaman *website* dan ketika diakses melalui URL, halaman dengan *phishing* tersebut bisa menyebabkan masalah bagi pengguna, mulai dari menginfeksi komputer pribadi dengan *malware*, mengunduh perangkat lunak mata-mata, mencuri kredensial, hingga pencurian uang dari kartu bank [3]. Salah satu cara paling baik untuk mengategorikan URL yang berbahaya atau tidak adalah dengan *Machine Learning*.

Jaringan Saraf Tiruan (Neural Network) sering disebut juga *Deep Learning* dan makin populer dalam penelitian saat ini khususnya dalam klasifikasi teks. *Neural Network* telah menunjukkan kinerja yang sangat baik dalam klasifikasi teks [4]. Bukti yang kuat dapat ditemukan melalui beberapa penelitian yang menunjukkan bahwa penggunaan *Neural Network* dapat menghasilkan akurasi yang sangat tinggi, bahkan melebihi 95% [5]. Sehingga *Neural Network* dapat digunakan untuk mendeteksi URL berbahaya dengan cukup akurat.

1.2. Rumusan Masalah

1. Bagaimana kombinasi layer LSTM dan CNN mampu untuk mengklasifikasikan url dengan baik?
2. *Variable* apa yang mempengaruhi akurasi dari model yang telah dibuat?
3. Kombinasi layer apa yang mampu mengidentifikasi URL dengan paling efektif?

1.3. Batasan Masalah

1. Metode yang digunakan adalah *Deep Learning Neural Network* dengan komposisi layer *word embedding*, *convolutional*, *max pooling*, LSTM(*Long Short-Term Memory*), kemudian dilakukan modifikasi *hyper-parameter* dan komposisi *layer* untuk mencari model dengan akurasi yang tinggi.
2. Data yang digunakan adalah data dari *website* pencatat *link* berbahaya atau *website* penyedia data dan kemudian dikombinasikan dengan

website yang masuk dalam *white list* atau *website* yang sering kita gunakan seperti google.com, youtube.com, dan sebagainya.

1.4. Tujuan Penelitian

1. Untuk mengetahui komposisi *layer Neural Network* apa yang dapat digunakan untuk mengklasifikasikan *URL* berbahaya dengan baik.
2. Untuk mengetahui *hyper-parameter* yang paling optimal untuk klasifikasi *URL*.

1.5. Manfaat Penelitian

1. Mengurangi ancaman kejahatan *cyber* bagi pengguna awam.
2. Peningkatan keamanan informasi *online*.

BAB II

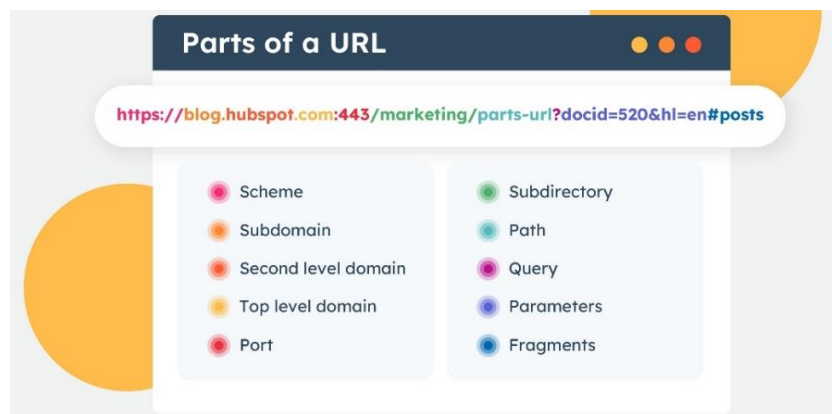
TINJAUAN PUSTAKA

Bab ini menjelaskan perihal teori teori yang digunakan sebagai landasan penelitian yang dikutip dari buku, publikasi dan penelitian yang relevan dengan apa yang sedang diteliti.

2.1. Landasan Teori

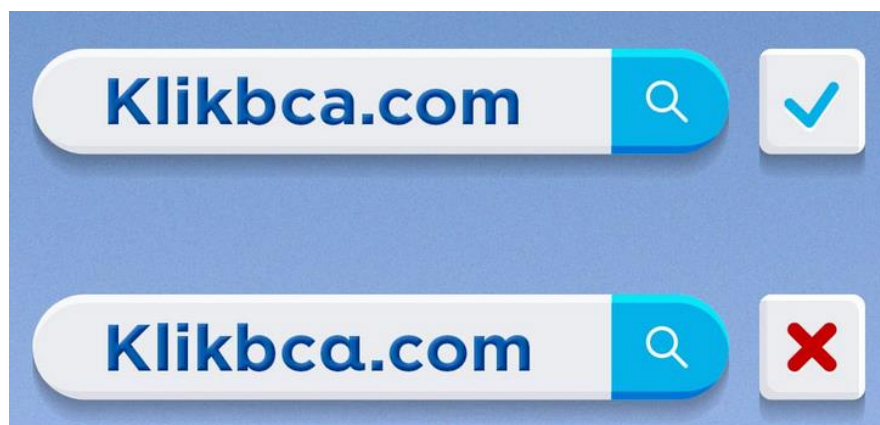
2.1.1. URL

URL (*Uniform Resource Locator*) atau tautan atau *link* adalah koneksi antara *resource* satu ke lainnya. Awal mula dari tautan adalah World Wide Web (WWW) yang berfungsi untuk melihat *multimedia-based* dokumen yang menampilkan dokumen dengan teks, gambar, animasi atau video. World Wide Web Consortium (W3C) adalah organisasi yang menstandarisasi perihal Hyper-Text Markup Language (XHTML), Cascading Style Sheets (CSS), HyperText Markup Language (HTML) dan Extensible Markup Language (XML). Untuk dapat melihat dokumen yang dipublikan dalam internet adalah dengan cara mengakses alamat URL-nya (URL address), struktur dasar dari URL terdiri dari protokol, *subdomain*, nama *domain*, dan ekstensinya[6].



Gambar 2.1.1 Struktur Tautan (<https://blog.hubspot.com/marketing/parts-url>)

URL yang berbahaya adalah URL yang dapat meminta pengguna internet untuk memberikan informasi penting dan/atau menggoda pengguna untuk mengaktifkan perintah komputer yang menginstal program yang dapat mencuri data pengguna. Biasanya, URL semacam itu memiliki alamat yang mirip (misalnya google.com dengan g00gle.com) dan antarmuka yang juga mirip. Pengguna yang tidak berhati-hati dapat tergoda untuk mengklik halaman tersebut dan mengisi informasi pribadi tanpa disadari.



Gambar 2.1.2 Contoh link phishing dengan membedakan font
(<https://www.bca.co.id/en/informasi/awas-modus/2023/06/26/07/22/waspada-website-palsu-menggunakan-abjad-karakter-yang-mirip-dengan-aslinya>)

2.1.2. Convolutional Neural Network

CNN adalah tipe lain dari *Deep Neural Network*, pada layer CNN proses yang dilakukan adalah mengekstrak fitur-fitur penting yang relevan dengan data sekarang sehingga matriks yang didapat lebih informatif dibandingkan sebelum masuk ke CNN. Mirip seperti CNN pada citra, CNN pada teks melakukan proses ekstraksi fitur dalam dimensi satu. Rumus yang digunakan adalah:

$$F(x) = (I * W)(x) = \sum_{i=1}^m I(x + i) \cdot W(i) \quad (2.1.1)$$

Di mana:

- $F(x)$ adalah nilai pada feature map.
- $I(x + i)$ adalah vektor kata pada posisi $x + i$.
- $W(i)$ adalah bobot filter pada posisi i .

Jika menggunakan CNN biasanya disertai juga penggunaan *pooling layer*, dimana *pooling layer* tersebut berfungsi untuk membagi region dari *convolutional* menjadi *subregion* agar komputasi menjadi lebih ringan. Peneliti menggunakan jenis max pooling, dimana mengambil data maksimum untuk merepresentasikan satu area filter [7]. Rumus yang digunakan adalah:

$$M(x) = \max (I(x + i)) \quad (2.1.2)$$

Di mana:

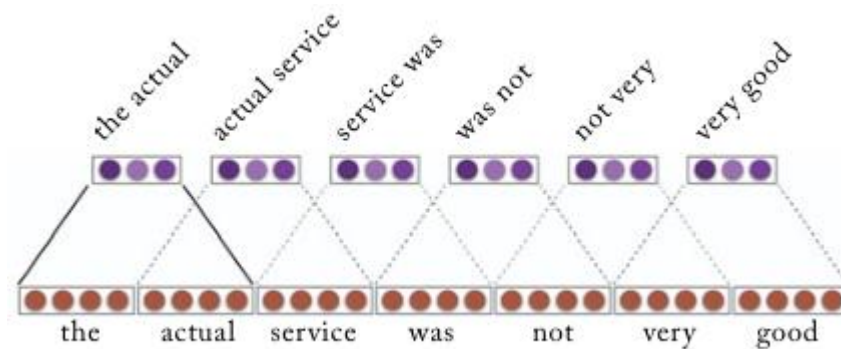
- $M(x)$ adalah nilai hasil max pooling pada posisi x .
- $I(x + i)$ adalah nilai vektor pada posisi $x + i$

CNN sangat sukses dalam kasus klasifikasi gambar [8] tetapi bisa juga digunakan dalam natural *language processing* [9].

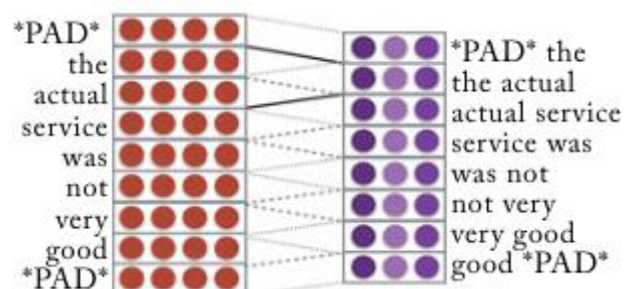
	Recurrent Network	Recursive Neural Tensor Network	Deep Belief Network	Convolution Network	MLP
Text Processing	✓	✓		✓	
Image Recognition			✓	✓	
Object Recognition		✓		✓	
Speech Recognition	✓				
Time Series Analysis	✓				
Classification			✓	✓	✓

Gambar 2.1.3 Use-case tiap Neural Network (T. Oliver, 2020)

Dalam klasifikasi teks, input akan diambil dari *embedding layer* kemudian di proses oleh *convolution layer* dengan perlakuan mirip seperti pemrosesan citra, yaitu dengan melakukan *filtering* pada tiap *window* terhadap *convolutional filter* (*kernel*) [10].



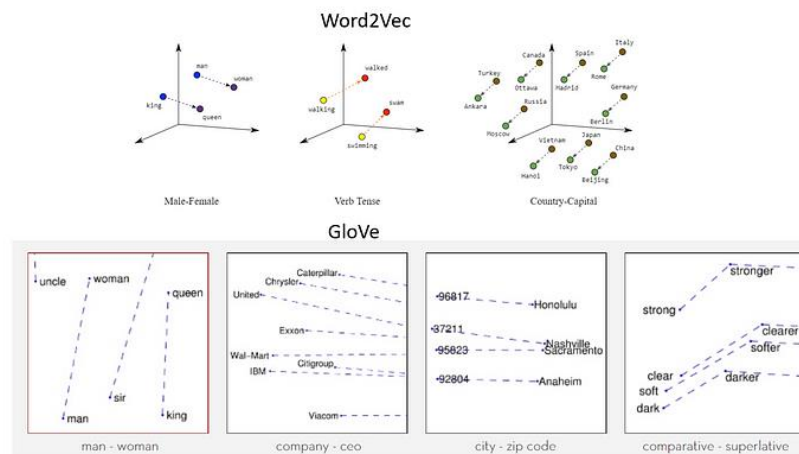
Gambar 2.1.4 CNN 1D (Graeme Hirst, dkk, 2017)



Gambar 2.1.5 CNN 1D 2 (Graeme Hirst, dkk, 2017)

2.1.3. Word Embedding

Word Embedding adalah Layer wajib untuk teks *processing*, karena pada layer ini tiap kalimat akan dikonversi menjadi vector dengan bobot angka yang dapat diproses oleh *computer* saat *training* [8].



Gambar 2.1.6 Ilustrasi Word Embedding (<https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4>)

2.1.4. Tokenizing

Tokenization adalah memecah kalimat menjadi kata kemudian melabeli kalimat tersebut dengan nomor agar dapat dikomputasikan [11]. Kemudian setelah dipecah pecah, nomor dari kata tersebut disusun ulang sesuai dengan susunan kalimat awalnya kedalam list (*teks to siquence*), setelah selesai kalimat kalimat dalam bentuk list tersebut di samakan dimensinya dan dimasukkan kedalam list yang sama (*padding*).

Output:

	I	enjoying	feel	for	freedom	happy	live	love	our	the	we
doc1	1	0	0	0	1	2	0	1	1	1	2
doc2	0	1	1	1	0	1	1	0	0	0	0

Gambar 2.1.7 Tokenizing (Alexandra George, 2018)

2.1.5. Long Short Term Memory (LSTM)

Long Short-Term Memory (LSTM) adalah varian dari RNN dan bekerja mirip dengan cara kerja RNN yang mengolah data sequensial atau time series, RNN biasa digunakan untuk memecahkan masalah ordinal atau temporal seperti *Natural Language Processing* (NLP), pengenalan suara, dan translasi bahasa, dan dapat kita temui dalam Google Translate, Google Assistant, pencarian menggunakan suara, dan Siri. Diciptakannya LSTM ditunjukan untuk mengatasi masalah vanishing gradient pada RNN (turunan dari fungsi aktivasi yang makin menurun ketika informasi melewati lapisan demi lapisan pada RNN) yang menyebabkan RNN sulit untuk mempelajari informasi dari masa lalu. LSTM memperkenalkan gate untuk mengatur aliran informasi yang relevan dengan rangkaian data yang sedang dilatih. Sel memori LSTM menyimpan bobot dari informasi sebelumnya dan mempengaruhi bobot output saat ini [12].

LSTM memiliki unit memori internal yang disebut cell state, unit ini dapat menyimpan informasi dari waktu sebelumnya dan memutuskan apakah informasi tersebut akan diteruskan atau dilupakan, LSTM juga memiliki tiga gerbang (gates): input gate, forget gate, dan output gate yang mengimplementasikan fungsi tulis, baca, dan reset dalam *cell memory* [8]. Gerbang-gerbang ini mengatur aliran informasi dalam dan keluar dari *cell state*. Rumus rumus dari LSTM adalah sebagai berikut:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.1.3)$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.1.4)$$

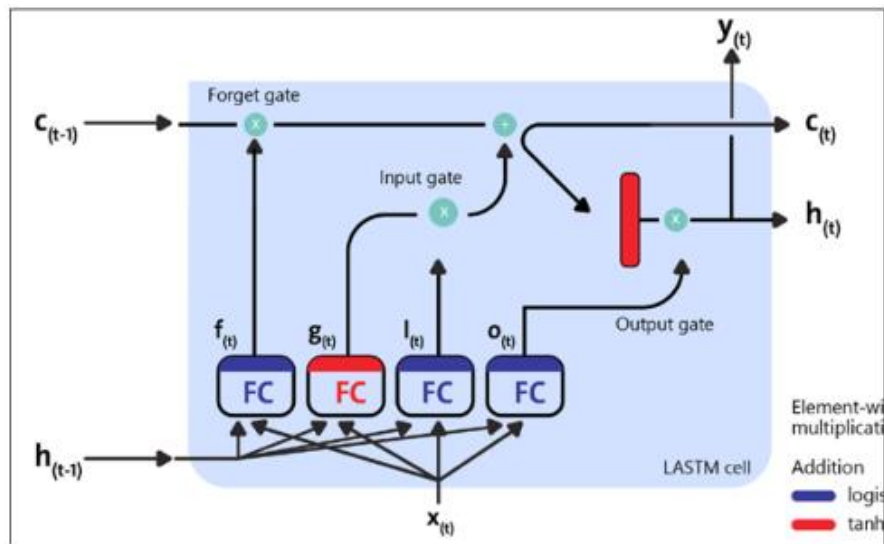
$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (2.1.5)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.1.6)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (2.1.7)$$

Di mana i_t adalah Input Gate, f_t adalah Forget Gate, C_t adalah Update Cell State, o_t adalah Output Gate, dan h_t adalah Hidden State, kemudian:

- x_t adalah input pada waktu t .
- h_{t-1} adalah hidden state pada waktu sebelumnya.
- W_i, W_f, W_c, W_o adalah matriks bobot.
- b_i, b_f, b_c, b_o adalah bias.
- σ adalah fungsi sigmoid.
- $\tanh$ adalah fungsi tangen hiperbolik.



Gambar 2.1.8 Arsitektur LSTM (G. Aurélien, 2019)

2.1.6. K-Fold Cross Validation

K-Fold Cross Validation adalah salah satu metode evaluasi kinerja model dengan cara membagi dataset menjadi k subset dengan ukuran yang sama, model dilatih pada k-1 subset dan diuji pada subset ke-k dan diulang sebanyak k, sehingga setiap subset sekali menjadi set uji. Performa K-Fold Cross Validation diukur dengan menghitung rata-rata dari hasil pengujian [13]. Rumus yang digunakan adalah:

$$Akurasi = \frac{Jumlah\ prediksi\ benar}{Total\ prediksi} \quad (2.1.8)$$

$$Presisi = \frac{TP}{TP + FP} \quad (2.1.9)$$

TP = banyak dari benar diprediksi benar

FP = banyak dari benar diprediksi salah.

$$Recall = \frac{TP}{TP + FN} \quad (2.1.10)$$

FN = banyak dari salah diprediksi benar.

Presisi dan *Recall* sering digabung menjadi satu metrik bernama *F1 Score* yang memberikan simplifikasi dengan *harmonic mean* dari *Presisi* dan *Recall* yang berarti *F1 Score* akan tinggi jika nilai dari *Presisi* dan *Recall* juga tinggi [14].

$$F1 = \frac{2 * TP * Recall}{2 * TP + Recall + Recall} \quad (2.1.11)$$

FP = banyak dari salah diprediksi salah.

2.2 Tinjauan Pustaka

Tabel 2.2.1 Tabel tinjauan pustaka

Peneliti	Judul	Hasil
Putra Wanda dan Huang Jin Jie	URLDeep: Continuous Prediction of Malicious URL with Dynamic Deep Learning in Social Networks	Hasil penelitiannya adalah bahwa pengembangan metode deteksi URL berbahaya yang menggunakan teknik deep learning berbasis LSTM dan transfer learning dengan fitur-fitur dinamis seperti yang disebutkan tersebut mampu mencapai tingkat akurasi yang tinggi dalam mendeteksi URL berbahaya secara kontinu dan dinamis di lingkungan jaringan sosial. Selain itu, metode ini juga mampu mengatasi permasalahan klasifikasi URL yang bersifat dinamis dan terus berubah seiring waktu. Walaupun demikian, tidak disebutkan berapa akurasi yang didapat

Zuguo Chen, Yanglong Liu, Chaoyang Chen, Ming Lu and Xuzhuo Zhang	Malicious URL Detection Based on Improved Multilayer Recurrent Convolutional Neural Network Model	Hasil penelitiannya adalah bahwa menggunakan metode yang disebutkan menghasilkan efektivitas dan kecepatan deteksi yang lebih baik dan mendapatkan high accuracy, high recall rate, and high accuracy dibanding dengan Text-RCNN, BRNN, dan model lainnya. Namun demikian masih diperlukan pengembangan untuk menghindari missing information dari URL teks vectorization. Akurasi dengan YOLO CSPDarknet menghasilkan 94% dengan nilai loss value 0.19. kemudian dengan traditional bidirectional recurrent neural network menunjukkan indikasi over fitting, dengan RCNN akurasinya adalah 92% dengan loss value 0.22, dengan tradisional RNN menghasilkan 90% dengan minimum loss value 0.32, dengan Neural Network models based on fully connected layers menghasilkan 86% dengan loss value 0.33.
Zhiqiang Wang, Xiaorui Ren, Shuhao Li, Bingyan Wang, Jianyi Zhang, dan Tao Yang	A Malicious URL Detection Model Based on Convolutional Neural Network	Penelitian ini bertujuan untuk merancang model deteksi URL berbahaya berbasis deep learning dengan menggunakan metode word embedding berdasarkan karakter embedding. Hasil eksperimen menunjukkan model deteksi yang dirancang efektif dalam mendeteksi URL berbahaya. Namun, dengan berkembangnya internet dan makin beragamnya jenis URL berbahaya, model ini harus diperbarui secara berkala agar tetap efektif dalam berbagai skenario aplikasi yang kompleks.

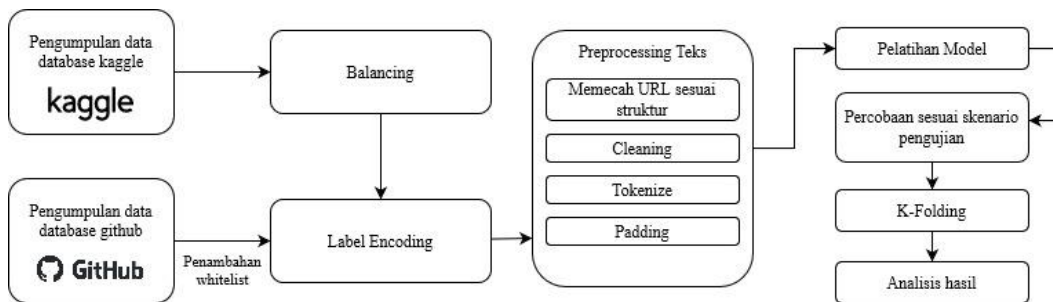
Huan-huan Wang & Long Yu & Sheng-wei Tian & Yong-fang Peng & Xin-jun Pei	Bidirectional LSTM Malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network	Hasil: Penelitian ini mendapatkan tingkat akurasi paling besar adalah 97.82%, tetapi setelah menggunakan URL word vector feature akurasi naik menjadi 98.45%. penggunaan algoritma CBIR yang menggabungkan fitur-fitur seperti texture fingerprint, URL word vector, dan static vocabulary dapat meningkatkan akurasi dalam mendeteksi halaman web yang jahat. Hal ini dapat diobservasi melalui hasil eksperimen yang menunjukkan bahwa metode yang diusulkan dalam penelitian ini memiliki hasil yang lebih baik dibandingkan dengan metode-metode lain yang telah diteliti sebelumnya.
Wei Wei, Qiao Ke, Jakub Nowak, Marcin Korytkowski, Rafał Scherer, Marcin Woźniak	Accurate and Fast URL Phishing Detector: A Convolutional Neural Network Approach	Kesimpulan dari penelitian ini adalah bahwa metode yang diusulkan dapat mengidentifikasi situs phishing dengan akurasi hampir 100% dan mampu mendeteksi serangan zero-day. Selain itu, metode ini lebih cepat dibandingkan dengan pendekatan lain yang menganalisis statistik lalu lintas atau konten web. Metode ini juga dapat digunakan pada perangkat mobile tanpa signifikan mempengaruhi kinerjanya. Jaringan saraf konvolusi yang diusulkan juga lebih mudah dilatih dan lebih cepat konvergensinya dibandingkan dengan jaringan LSTM yang dibandingkan dalam penelitian sebelumnya.
Andrei Cristian, Gabriel Florea, Lorand Halasz, Camelia Lemnaru, Ciprian Oprisa	Detecting Malicious URLs Based on Machine Learning Algorithms and Word Embeddings	Kombinasi antara word embeddings dan fitur-fitur domain yang telah dirancang khusus, Teknik oversampling sintetis dan cost-sensitive learning digunakan untuk mengatasi ketidakseimbangan kelas. Teknik-teknik klasifikasi yang berbeda juga dieksplorasi dalam penelitian ini (Cost-Sensitive NN, MLP, Extra Trees).

BAB III

METODE PENELITIAN

Bab ini berisi pembahasan mengenai gambaran umum penelitian, deskripsi data, langkah – langkah dalam penelitian, skenario pengujian, kebutuhan hardware dan software, serta jadwal penelitian.

3.1. Gambaran Umum Penelitian



Gambar 3.1.1 Diagram alur penelitian

Berdasarkan diagram pada Gambar 3.1.1, penelitian ini dimulai dengan mengumpulkan data dari Kaggle. Selanjutnya, dilakukan tahap Exploratory Data Analysis (EDA). Jika ditemukan ketidakseimbangan data antar kelas, maka diterapkan proses balancing pada data. Setelah itu, dilakukan penambahan whitelist dari repositori GitHub yang dilabeli sebagai benign. Setelah selesai, dilakukan label encoding agar label dapat diproses oleh komputer.

Selanjutnya, data URL dipecah menjadi bagian-bagian sesuai struktur dari URL, yaitu scheme, subdomain, second-level domain, top-level domain, subdirectory, query, dan path. Setelah data terpecah, URL dibersihkan dari simbol-simbol dan dilakukan tokenisasi serta padding pada data yang sudah bersih tersebut.

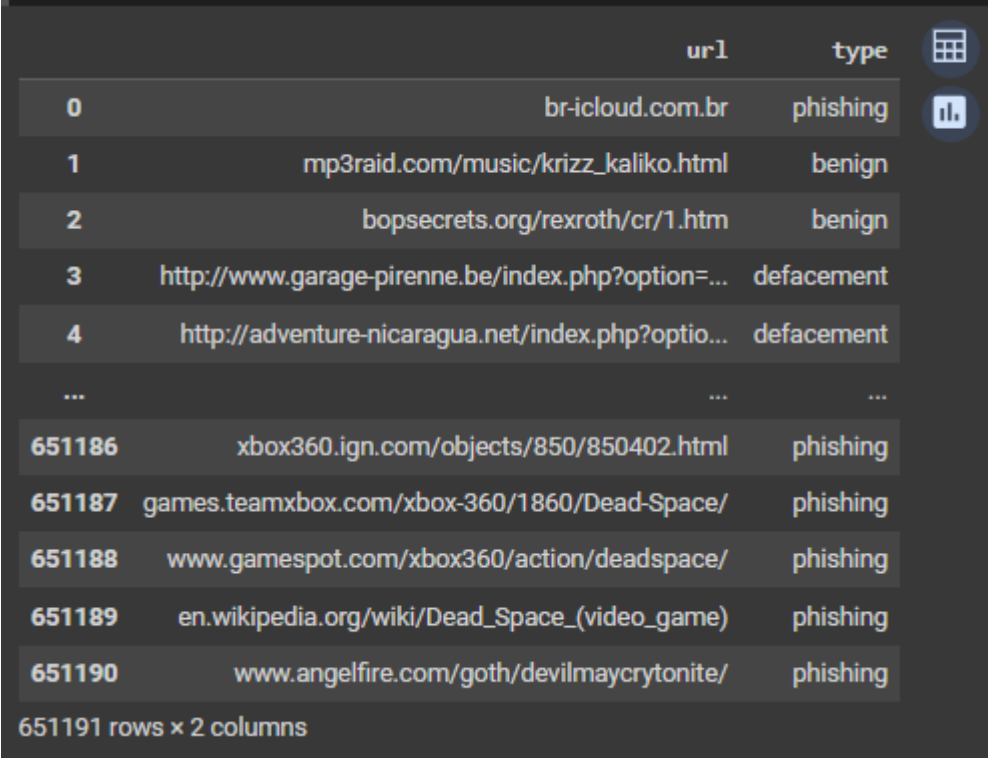
Pada tahapan pelatihan model, data diuji dengan beberapa kombinasi hyperparameter untuk melihat mana yang memberikan akurasi terbaik menggunakan metode K-Fold Cross Validation.

3.2. Deskripsi Data

Dalam penelitian ini, dataset URL diambil dari Kaggle dan ditambah dengan data white list dari Phising Database milik mitchellkrogza. Data utama adalah data dari Kaggle karena data tersebut sudah dikurasi dari beberapa sumber data kolektif dari Perusahaan (Canadian Institute for Cybersecurity, PishTank, PishStrom), sedangkan dataset dari mitchellkrogza divalidasi oleh tim mereka, sehingga data dari Kaggle dirasa lebih reliable untuk digunakan sebagai data inti. Berikut adalah data dari Kaggle dan whitelist dari mitchellkrogza:

	url	type
0	100realt.ru	benign
1	104.219.234.142	benign
2	1040taxbiz.com	benign
3	1drv.ms	benign
4	213.238.34.213	benign
...	...	...
597	docs.google.com	benign
598	drive.google.com	benign
599	firebasestorage.googleapis.com	benign
600	googleusercontent.com	benign
601	onedrive.live.com	benign

Gambar 3.2.1 Dataset whtielist dari GitHub



	ur1	type
0	br-icloud.com.br	phishing
1	mp3raid.com/music/krizz_kaliko.html	benign
2	bopsecrets.org/rexroth/cr/1.htm	benign
3	http://www.garage-pirenne.be/index.php?option=...	defacement
4	http://adventure-nicaragua.net/index.php?optio...	defacement
...	...	...
651186	xbox360.ign.com/objects/850/850402.html	phishing
651187	games.teamxbox.com/xbox-360/1860/Dead-Space/	phishing
651188	www.gamespot.com/xbox360/action/deadspace/	phishing
651189	en.wikipedia.org/wiki/Dead_Space_(video_game)	phishing
651190	www.angelfire.com/goth/devilmaycrytonite/	phishing

651191 rows x 2 columns

Gambar 3.2.2 Dataset dari Kaggle

Whitelists yang digunakan adalah whitelist di tanggal 10 Agustus 2023 di jam 18.00, dan data dari Kaggle adalah data versi terakhir dari repository-nya.

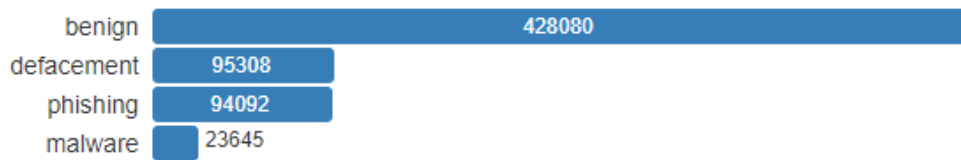
3.3. Tahapan Penelitian

Tahapan penelitian berisi langkah – langkah yang dilalui dalam melakukan penelitian seperti pada diagram alur pada sub bab gambaran umum penelitian, serta menjelaskan lebih detail pada tiap tahapannya.

3.3.1. Exploratory Data

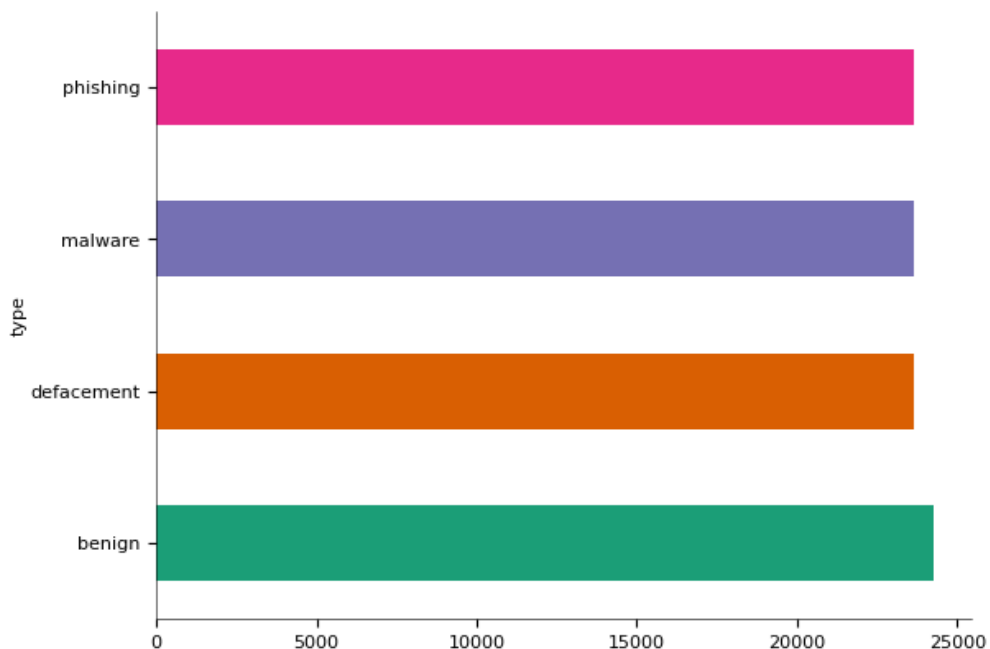
Data yang diambil berupa teks URL dari Kaggle dan GitHub dengan data sebanyak 641.125 dan ketika diberlakukan drop duplikasi menjadi 641.119 yang

berisika empat kelas yaitu benign, defacement, phishing, malware. Kemudian data white list dari GitHub sebanyak 602 yang berisikan satu kelas yaitu benign.



Gambar 3.3.1 Persebaran Data

Tahapan pemrosesan pertama yang diberlakukan pada data adalah balancing data, dikarena data yang didapat tidak seimbang maka dilakukan penyeimbangan data menggunakan metode under-sampling dengan mengambil sampel sebanyak data paling sedikit yaitu sebanyak 23.645 data tiap kelas, setelah dilakukan balancing data yang sudah balance ditambah dengan data whitelist dari GitHub dengan data sebanyak 602, sehingga menghasilkan data balance dengan sebaran data sebagai berikut:



Gambar 3.3.2 data setelah balancing dan ditambah whitelist

[illegible]

Gambar 3.3.4 Wordcloud data defacement



Gambar 3.3.7 Wordcloud data malware

Dapat dilihat pada wordcloud data malware, banyak mengarah ke file exe executable file untuk menargetkan pengguna windows, kemudian mozi.m botnet script DDoS untuk menargetkan perangkat IoT dan juga ada sebagian kecil diarahkan ke file .sh bash scrip untuk pengguna linux.

3.3.2. Label Encoding

Proses encoding label dilakukan dengan cara mencatat berapa banyak nilai unik dari sebuah kolom, kemudian melakukan enumerasi untuk membuat dictionary secara otomatis. Hasilnya adalah sebagai berikut: `{'benign': 0, 'defacement': 1, 'phishing': 2, 'malware': 3}`.

Setelah dictionary terbentuk, proses mapping dilakukan pada kolom target untuk mengubah string label menjadi numerical label. Berikut adalah kode yang digunakan:

```
# ubah label jadi angka
def labeling_data(data):
    mapping_counters = {}
    value_counts = data['type'].value_counts()
    category_mapping = {category: int(idx) for idx, category in enumerate(value_counts.index)}
    mapping_counters['type'] = category_mapping

    data['type'] = data['type'].map(category_mapping)
    return category_mapping
```

Gambar 3.3.8 Blok Kode untuk label encoding

3.3.3. Cleaning dan Normalize

Proses pembersihan data pertama yang diterapkan adalah mengubah semua huruf dalam data menjadi huruf kecil. Langkah ini penting untuk menghindari bias antara huruf besar dan kecil saat melakukan pelatihan model. Selanjutnya, simbol-simbol umum dalam URL seperti titik dua (:), garis miring (/), titik (.), koma (,), dan simbol lain yang termasuk dalam tanda baca dihilangkan. Berikut adalah implementasi kode untuk melakukan dua proses tersebut:

```
# Convert to lowercase
token = token.lower()
# Remove punctuation and special characters
token = ''.join(char for char in token if char not in string.punctuation)
cleaned_tokens.append(token)
```

Gambar 3.3.9 Blok Kode untuk cleaning

Dalam implementasi ini, penulis memanfaatkan daftar punctuation yang telah disediakan oleh modul string untuk menghilangkan karakter yang termasuk dalam tanda baca dari setiap string yang telah ditokenisasi.

```
1 import string
2 result = string.punctuation
3 print(result)

!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Gambar 3.3.10 Daftar punctuation dalam library String bawaan python

3.3.4. Tokenize

Tokenisasi dilakukan dengan memanfaatkan paket modul standar python `urllib.parse`. Langkah pertama yang dilakukan adalah dengan melakukan parsing atau memecah URL berdasarkan komponen komponennya, yaitu skema, domain, path, query.

```
def tokenize_url(url):
    # Parse the URL
    parsed_url = urlparse(url)
```

Gambar 3.3.11 Blok kode untuk memecah url

Selanjutnya, mengambil data komponen yang sudah diparsing diambil dan setiap kata di dalamnya diekstrak menggunakan *regex*, lalu dimasukkan ke dalam sebuah *array*.

```
# Extract components
scheme = parsed_url.scheme
domain = parsed_url.netloc
path = parsed_url.path

# Tokenize the scheme, domain, and path
scheme_tokens = re.findall(r'\w+', scheme) if scheme else []
domain_tokens = re.findall(r'\w+', domain) if domain else []
path_tokens = re.findall(r'\w+', path) if path else []
```

Gambar 3.3.12 Blok kode untuk memasukkan url ke masing masing array

Proses serupa juga dilakukan pada komponen query, di mana *regex* digunakan untuk mengekstrak kata-kata dan memasukkannya ke dalam *array*.

```
# Tokenize query parameters
query_params = parse_qs(parsed_url.query)
query_tokens = []
for key, values in query_params.items():
    query_tokens.extend(re.findall(r'\w+', key))
    for value in values:
        query_tokens.extend(re.findall(r'\w+', value))
```

Gambar 3.3.13 Blok kode penggabungan kembali

Setelah semua proses yang telah dijelaskan selesai, data dari skema, domain, path dan query yang telah ditokenisasi digabungkan menjadi satu kedalam sebuah array. Array hasil penggabungan tersebut selanjutnya dikembalikan sebagai nilai balik dari fungsi “tokenize_url”.

Berikut ini merupakan ilustrasi penggunaan fungsi tokenize_url dengan menggunakan URL berikut sebagai contoh:

‘https://pages.store.office.com/addinsinstallpage.aspx?assetid=WA104382081&rs=en-US&correlationId=88d8fa8c-600f-36f3-eb63-8be5e9081364.’

Pertama, pemecahan URL ke masing masing bagian. Hasil pemecahan URL adalah sebagai berikut:

- Skema: ‘https’
- Domain: ‘pages.store.office.com’
- Path: ‘/addinsinstallpage.aspx’
- Query: ‘assetid=WA104382081&rs=en-US&correlationId=88d8fa8c-600f-36f3-eb63-8be5e9081364’

Kemudian, skema, domain, dan path ditokenisasi. Hasil tokenisasi adalah sebagai berikut:

- Token skema: ['https']
- Token domain: ['pages', 'store', 'office', 'com']
- Token path: ['addinsinstallpage', 'aspx']

Selanjutnya, parameter query ditokenisasi. Hasil tokenisasi parameter query adalah sebagai berikut:

- Token query: ['assetid', 'WA104382081', 'rs', 'en', 'US', 'correlationId', '88d8fa8c', '600f', '36f3', 'eb63', '8be5e9081364']

Menghasilkan:



```

Tokens for URL (erickson.edu/) 1:
['erickson', 'edu']

Tokens for URL (crosstitch.com/search?f=lib01) 2:
['crosstitch', 'com', 'search', 'f', 'lib01']

Tokens for URL (http://jalopnik.com/apple-wants-to-make-navigation-like-a-friend-reading-vo-1700721307/layer/) 3:
['http', 'jalopnik', 'com', 'apple', 'wants', 'to', 'make', 'navigation', 'like', 'a', 'friend', 'reading', 'yo', 'layer']

Tokens for URL (hpieurope.com/kit-info.php?partNo=102852&lang=en) 4:
['hpieurope', 'com', 'kit', 'info', 'php', 'partno', '102852', 'lang', 'en']

Tokens for URL (youtube.com/watch?v=4nxZfgnGHa0) 5:
['youtube', 'com', 'watch', 'v', '4nxZfgnGHa0']

Tokens for URL (projects.latimes.com/hollywood/star-walk/jo-van-fleet/) 6:
['projects', 'latimes', 'com', 'hollywood', 'star', 'walk', 'jo', 'van', 'fleet']

Tokens for URL (encyclopedia2.thefreedictionary.com/Iraq+war+(disambiguation))) 7:
['encyclopedia2', 'thefreedictionary', 'com', 'iraq', 'war', 'disambiguation']

```

Gambar 3.3.14 Hasil tokenisasi

Kemudian berdasarkan list diatas, dalam tiap kalimat yang unik akan diwakilkan dengan angka yang unik, jika sama maka akan diwakilkan dengan angka yang sama.

```

# Tokenize the URLs
tokenizer = Tokenizer()
tokenizer.fit_on_texts(tokenized_urls)
sequences = tokenizer.texts_to_sequences(tokenized_urls)

# Pad the sequences to a fixed length
X_padded = pad_sequences(sequences, maxlen=max_sequence_length)

y_encoded = to_categorical(labels)

word_index = tokenizer.word_index
print('Found %s unique tokens.' % len(word_index))

```

Found 116268 unique tokens.

Gambar 3.3.15 blok kode tokenisasi menggunakan library

Pada data peneliti terdapat 116.268 ribu token untuk dilatih. Dan barulah data dapat dimasukan kedalam model untuk dilatih.

3.3.5. Layer Embedding

Pada layer ini, kumpulan data hasil tokenisasi diproses menjadi vektor embedding berdimensi tertentu, tergantung dengan nilai parameter output shapenya. Bobot vektor awal akan diinisialisasi secara acak dan akan diperbaharui selama proses pelatihan. Contoh ini siasi bobot vektor dan dimensi vektor sebagai berikut:

```
['https', 'pages', 'store', 'office', 'com', 'addinsinstallpage', 'aspx', 'assetid',
'WA104382081', 'rs', 'en', 'US', 'correlationId', '88d8fa8c', '600f', '36f3', 'eb63',
'8be5e9081364']
```

```
<tf.Tensor: shape=(1, 18, 2), dtype=float32, numpy=
array([[[ -0.01908034,  0.03861847],
         [ 0.03995821, -0.03053806],
         [-0.04120221, -0.01564108],
         [-0.02079078,  0.01690531],
         [ 0.01385763, -0.03331822],
         [-0.01176133, -0.00384615],
         [ 0.00248452,  0.02336382],
         [ 0.02789558, -0.02883978],
         [ 0.03789785,  0.00427191],
         [-0.01920336,  0.02379079],
         [-0.02115351,  0.01275298],
         [ 0.0332779 ,  0.00946003],
         [ 0.03966541, -0.00348798],
         [-0.04169518, -0.01172306],
         [-0.04120693, -0.00742741],
         [ 0.04015583,  0.01533734],
         [-0.03361259,  0.01022542],
         [ 0.00825135, -0.01846467]]], dtype=float32)>
```

Gambar 3.3.16 output neuron 2

```
<tf.Tensor: shape=(1, 18, 4), dtype=float32, numpy=
array([[[ 0.02210157, -0.01442808,  0.02508466,  0.04504886],
         [-0.02520921, -0.03011799, -0.02329307, -0.04766263],
         [ 0.03661301,  0.022317 , -0.04105455,  0.04243096],
         [-0.01488104,  0.0186 ,  0.02535516,  0.00488672],
         [ 0.02387745,  0.01679016, -0.02364152,  0.0060959 ],
         [-0.00167899,  0.03132757,  0.03962976, -0.03286178],
         [-0.02054981, -0.00757687,  0.01114387, -0.01254107],
         [-0.02406098, -0.02980325,  0.04636644, -0.04492496],
         [ 0.02641353,  0.00453936, -0.0035538 ,  0.03738636],
         [ 0.01614766,  0.00017399,  0.03473053, -0.00820721],
         [ 0.01185832,  0.0079677 , -0.02548361,  0.01163862],
         [ 0.0138914 ,  0.04237558, -0.00754281, -0.0149905 ],
         [-0.00262392, -0.01472669, -0.0203277 ,  0.0456812 ],
         [-0.01813231,  0.0473469 ,  0.02314356, -0.02567519],
         [-0.011436 , -0.0248998 ,  0.01269251, -0.02137074],
         [-0.04887975, -0.03286751,  0.04081466,  0.00357535],
         [-0.0179581 ,  0.03012378, -0.00155519,  0.03977081],
         [-0.04195763,  0.03917981,  0.03344394,  0.00230271]]],
      dtype=float32)>
```

Gambar 3.3.17 output neuron 4

3.3.6. Layer Convolutional Neural Network

Contoh pengerjaan CNN pada tensorflow jika kita memiliki matriks vektor $I = [3,1,4,2,5]$, kemudian menerapkan CNN satu dimensi pada vektor tersebut dengan filter 2×1 : $g(x) = [\frac{1}{2}, \frac{1}{2}]$. maka menghasilkan:

$$h(1) = 3 * \frac{1}{2} + 1 * \frac{1}{2} = 2$$

$$h(2) = 1 * \frac{1}{2} + 4 * \frac{1}{2} = 2.5$$

$$h(3) = 4 * \frac{1}{2} + 2 * \frac{1}{2} = 3$$

$$h(4) = 2 * \frac{1}{2} + 5 * \frac{1}{2} = 3.5$$

Sehingga menghasilkan matriks vektor $I = [2, 2.5, 3, 3.5]$

Contoh pengerjaan layer MaxPooling tensorflow, jika kita memiliki matriks vektor $I = [3, 1, 4, 2, 5]$, kemudian menerapkan maxpooling pada vektor tersebut maka menghasilkan:

$$M(1) = \max(3, 1) = 3$$

$$M(2) = \max(1, 4) = 4$$

$$M(3) = \max(4, 2) = 4$$

$$M(4) = \max(2, 5) = 5$$

Sehingga menghasilkan matriks vektor $I = [3, 4, 4, 5]$

Neuron yang divariasikan dalam program peneliti adalah adalah [32, 64, 128, 256, 512]

```
<tf.Tensor: shape=(1, 8, 4), dtype=float32, numpy=
array([[0.03473107, 0.02187518, 0.01246977, 0.04695098],
       [0.01907397, 0.01732575, 0.04330425, 0.
       ],
       [0.0511762 , 0.01654365, 0.008924 , 0.03912671],
       [0.01434586, 0.02129882, 0.04663947, 0.
       ],
       [0.05372129, 0.
       , 0.07606247, 0.02438662],
       [0.00436814, 0.
       , 0.02430429, 0.04837126],
       [0.05776923, 0.02574828, 0.01425277, 0.05624728],
       [0.02372896, 0.
       , 0.00107041, 0.03720537]]], dtype=float32)>
```

Gambar 3.3.18 CNN neuron 4

3.3.7. Optimalisasi Resource Hardware

```
delete_variables([
    'model', 'X_train_fold',
    'X_val_fold', 'y_train_fold',
    'y_val_fold', 'train_generator',
    'val_generator', 'end_time',
    'start_time', 'time_used'
])
```

Gambar 3.3.19 Blok kode optimalisasi

Blok kode optimisasi RAM untuk mencegah terjadinya kegagalan sistem (crash) selama proses iterasi K-Fold. Blok kode ini beroperasi dengan mengeliminasi variabel-variabel yang tidak lagi diperlukan dan akan ditugaskan ulang pada iterasi berikutnya, sehingga meminimalisir penggunaan memori yang berlebihan dan meningkatkan efisiensi dalam pengelolaan sumber daya komputasi. Penggunaan cara ini menghasilkan penurunan penggunaan ram yang cukup signifikan.

Tabel 3.3.1 Uji optimalisasi resource pertama

fold	iterasi	tanpa optimisasi	dengan optimisasi
5 Fold	iterasi 1	1342	1346
	iterasi 2	1518	2,388
	iterasi 3	1654	3,466
	iterasi 4	1816	4,520
	iterasi 5	1972	5,580
rata rata perubahan		78.75 mb	157.5 mb

Tabel 3.3.2 Uji optimalisasi resource kedua

fold	iterasi	optimisasi	Tanpa optimisasi
5 Fold	iterasi 1	1318	1340
	iterasi 2	1486	2,542
	iterasi 3	1622	3,766
	iterasi 4	1780	4976
	iterasi 5	1932	6,188
rata rata perubahan		153.5mb	1212mb

3.3.8. Evaluasi

Evaluasi performa model komputasi dilakukan dengan teknik Cross Validation K-Fold, di mana parameter K ditetapkan dengan nilai minimum 3 dan maksimum 15. Proses iterasi akan berlangsung secara berkelanjutan hingga ditemukan kesalahan (error) atau tercapainya batas maksimum K yang telah ditentukan.

```
# Iterate over different fold values
def train_using_fold(fold_values = 3, threshold = 15):
    while True:
        if fold_values > threshold:
            break
```

Gambar 3.3.20 Blok kode fungsi folding

```
for index, val_index in enumerate(kf.split(X_padded, y_single)):
    X_train_fold, X_val_fold = X_padded[val_index[0]], X_padded[val_index[1]]
    y_train_fold, y_val_fold = y_encoded[val_index[0]], y_encoded[val_index[1]]

    model = tuner.hypermodel.build(best_hps)
```

Gambar 3.3.21 Blok kode perulangan dalam satu fold

3.4. Skenario Pengujian

Tabel 3.4.1 Skenario Pengujian Dengan K-Fold dan Tanpa K-Fold

Skenario	Layer	Learning rate	Neuron
1	4 CNN	1e-5	[32, 64, 128, 256]
2	4 CNN	1e-4	[32, 64, 128, 256]
3	4 CNN	1e-3	[32, 64, 128, 256]

4	4 LSTM	1e-5	[32, 64, 128, 256]
5	4 LSTM	1e-4	[32, 64, 128, 256]
6	4 LSTM	1e-3	[32, 64, 128, 256]
7	1 CNN, 3 LSTM	1e-5	[32, 64, 128, 256]
8	1 CNN, 3 LSTM	1e-4	[32, 64, 128, 256]
9	1 CNN, 3 LSTM	1e-3	[32, 64, 128, 256]
10	2 CNN, 2 LSTM	1e-5	[32, 64, 128, 256]
11	2 CNN, 2 LSTM	1e-4	[32, 64, 128, 256]
12	2 CNN, 2 LSTM	1e-3	[32, 64, 128, 256]
13	3 CNN, 1 LSTM	1e-5	[32, 64, 128, 256]
14	3 CNN, 1 LSTM	1e-4	[32, 64, 128, 256]
15	3 CNN, 1 LSTM	1e-3	[32, 64, 128, 256]

Untuk pengujian dengan metode K-Fold, peneliti menerapkan hingga 15 lipatan (fold) atau hingga terjadi kegagalan (crash). Sementara itu, untuk pengujian tanpa metode K-Fold, diambil akurasi model dalam satu kali eksekusi. Langkah selanjutnya adalah memilih akurasi terbaik dari skenario tanpa K-Fold dan menghitung rata-rata akurasi terbaik dari skenario dengan K-Fold.

3.5. Kebutuhan Perangkat Hardware dan Software

a. Spesifikasi Hardware

- i. Intel(R) Core i5-6330U @ 2.40GHz (4 CPUs) vPro
- ii. RAM 8 GB DDR4 Singel-Channel 2133 Mhz
- iii. Intel(R) HD Graphics 520
- iv. SSD

b. Spesifikasi Software

- i. Sistem Operasi Windows 10 Pro 64-bit
- ii. Browser
- iii. Kaggle Notebook (Jupyter)
- iv. Python 3.*
- v. Library Python;
 - TensorFlow dan Keras
 - Pandas
 - Sklearn
 - Numpy
 - Matplotlib

BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

Bab ini membahas mengenai hasil skenario pengujian dari penelitian dan analisis hasil pengujian dari penelitian.

4.1 Hasil Skenario Pengujian

Tabel 4.1.1 Hasil Uji dengan K-Fold

Layer	Learning rate	Neuron	Akurasi
CNN(2), LSTM(2)	0.001	128, 64 & 128, 128	0.82880
CNN(0), LSTM(4)	0.0001	128, 256, 256, 256	0.88647
	0.001	256, 256, 128, 128	0.91724
		128, 256, 128, 128	0.91800
		128, 128, 128, 128	0.91928
		128, 128, 128, 128	0.92388
		128, 256, 128, 256	0.92393
		256, 128, 256, 128	0.92718
		128, 128, 256, 256	0.92827
		128, 128, 128, 128	0.92990
CNN(1), LSTM(3)	0.0001	128 & 128, 256, 128	0.89054
		128 & 128, 128, 128	0.89115
		256 & 128, 256, 128	0.89967
	0.001	128 & 128, 256, 256	0.90714
		128 & 128, 128, 128	0.91115
		128 & 128, 256, 256	0.91500
		128 & 128, 64, 64	0.91501
		256 & 128, 256, 256	0.91700
		128 & 128, 128, 128	0.91760
		128 & 256, 128, 128	0.91948
		256 & 128, 256, 128	0.92108
		128 & 128, 128, 128	0.92302

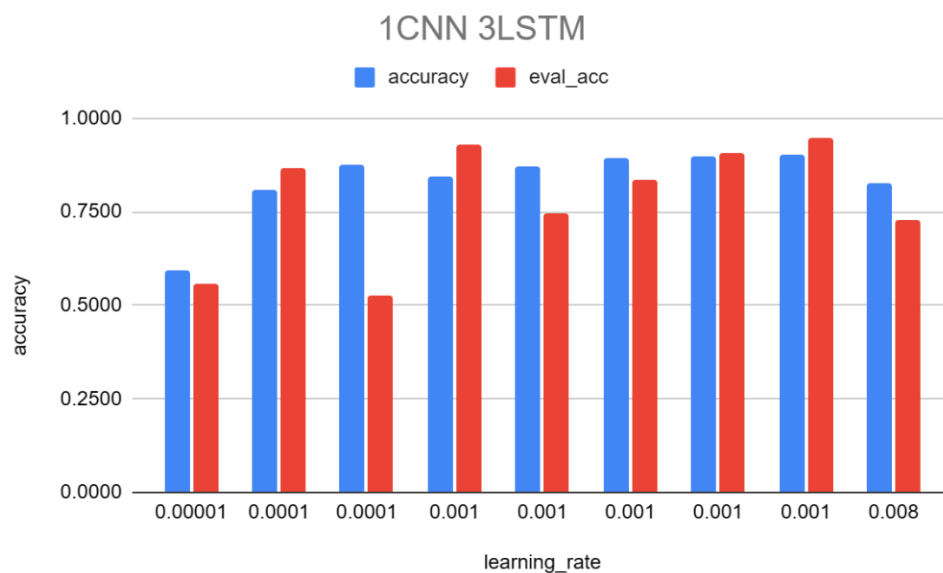
Tabel 4.1.2 Hasil Uji tanpa K-Fold

Layer	Learning rate	Neuron	Akurasi
CNN(2), LSTM(2)	0.001	256, 128 & 128, 128	0.8706
		256, 128 & 128, 128	0.8600
		256, 128 & 128, 128	0.8549
		256, 128 & 128, 128	0.8459
		256, 128 & 128, 128	0.7440
		256, 128 & 128, 128	0.7422
	0.000001	256, 128 & 128, 128	0.7405

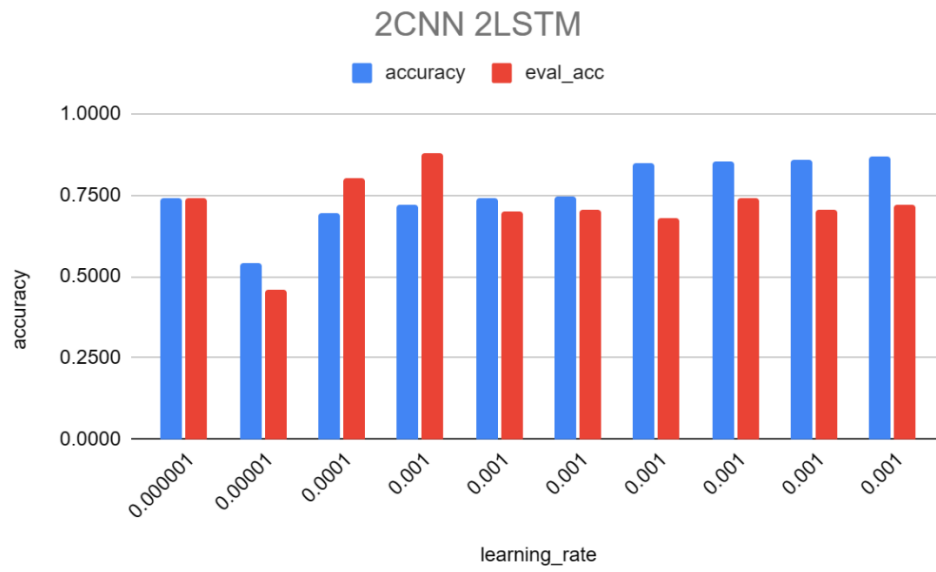
CNN(1), LSTM(2)	0.001	256 & 128, 128	0.8647
		256 & 128, 128	0.8602
		256 & 128, 128	0.8513
		256 & 128, 128	0.4966
CNN(1), LSTM(3)	0.001	256 & 256, 128, 128	0.9005
		256 & 256, 128, 128	0.8965
		256 & 256, 128, 128	0.8937
		256 & 128, 128, 128	0.8695
	0.0001	256 & 256, 128, 128	0.8736
	0.008	256 & 256, 128, 128	0.8269
CNN(4), LSTM(0)	0.001	256, 128, 128, 128	0.3000
	0.0001	32, 32, 128, 64	0.2943
	0.00001	64, 128, 256, 32	0.2882
CNN(0), LSTM(4)	0.001	256, 128, 128, 128	0.8706
	0.0001	64, 512, 512, 32	0.8071
	0.00001	512, 256, 512, 128	0.5515
	0.001	128, 128, 64, 32	0.9048
CNN(3), LSTM(1)	0.001	256, 128, 128 & 128	0.7473
	0.0001	64, 256, 128 & 256	0.3929
	0.00001	256, 512, 128 & 32	0.3722

4.2 Analisa Hasil Pengujian

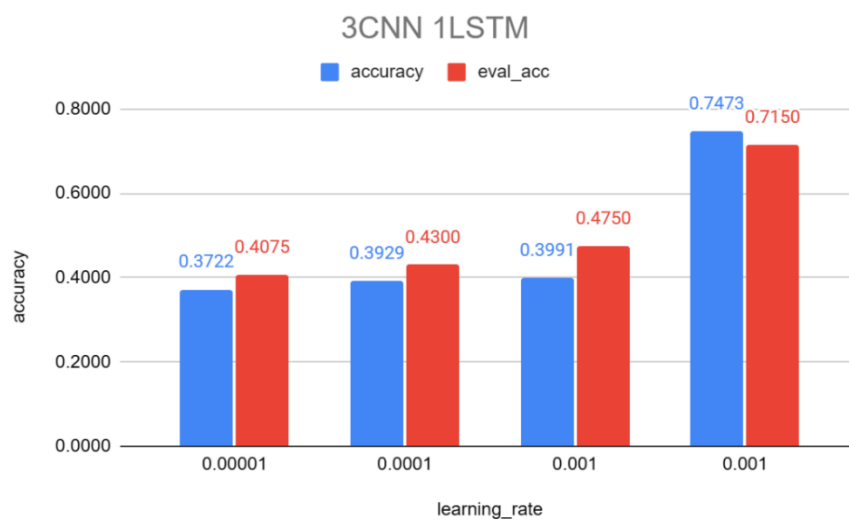
4.2.1 Pengujian Tanpa K-Fold



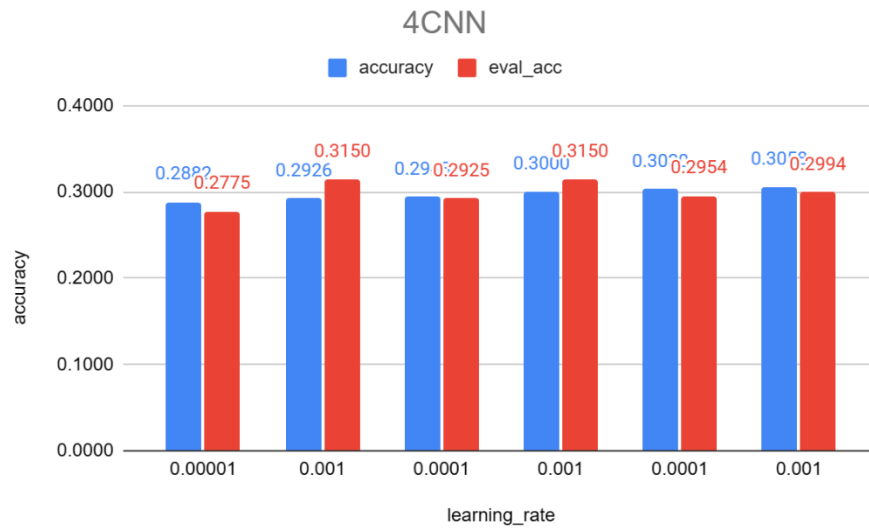
Gambar 4.2.1 Grafik Akurasi Arsitektur 3LSTM 1CNN



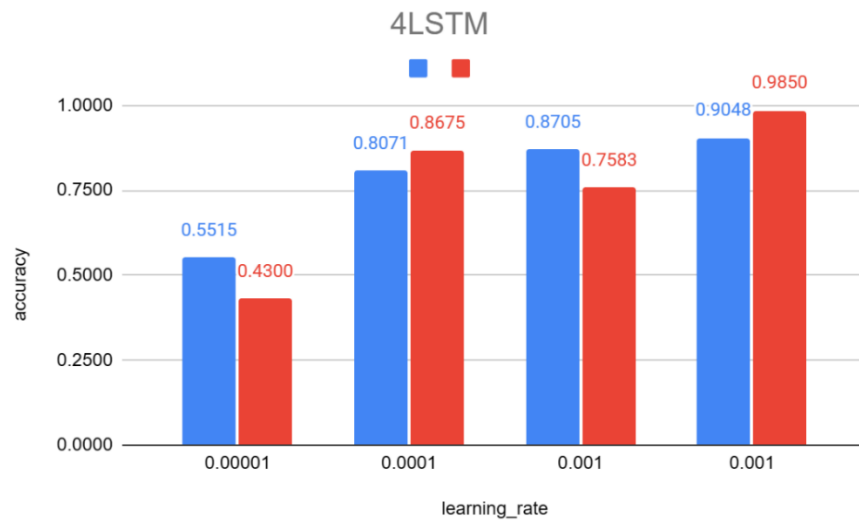
Gambar 4.2.2 Grafik Akurasi Arsitektur 2LSTM 2CNN



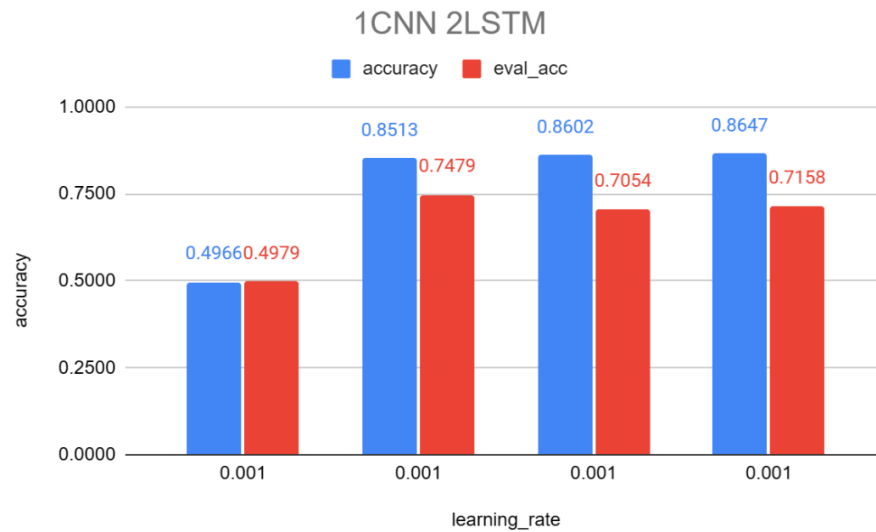
Gambar 4.2.3 Grafik Akurasi Arsitektur 1LSTM 3CNN



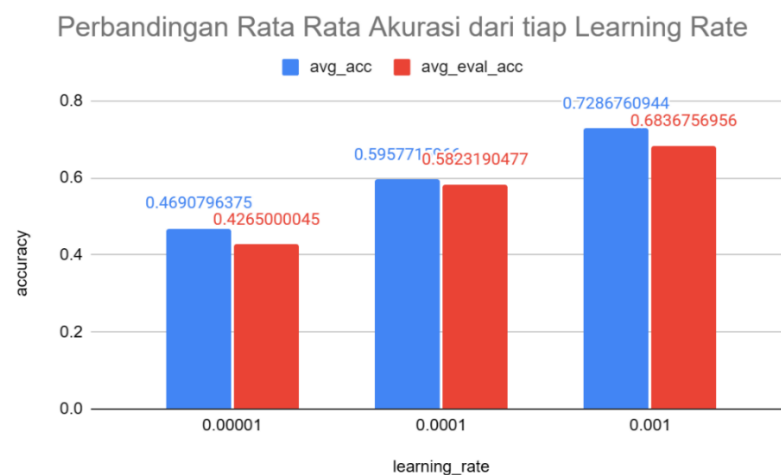
Gambar 4.2.4 Grafik Akurasi Arsitektur 4CNN



Gambar 4.2.5 Grafik Akurasi Arsitektur 4LSTM



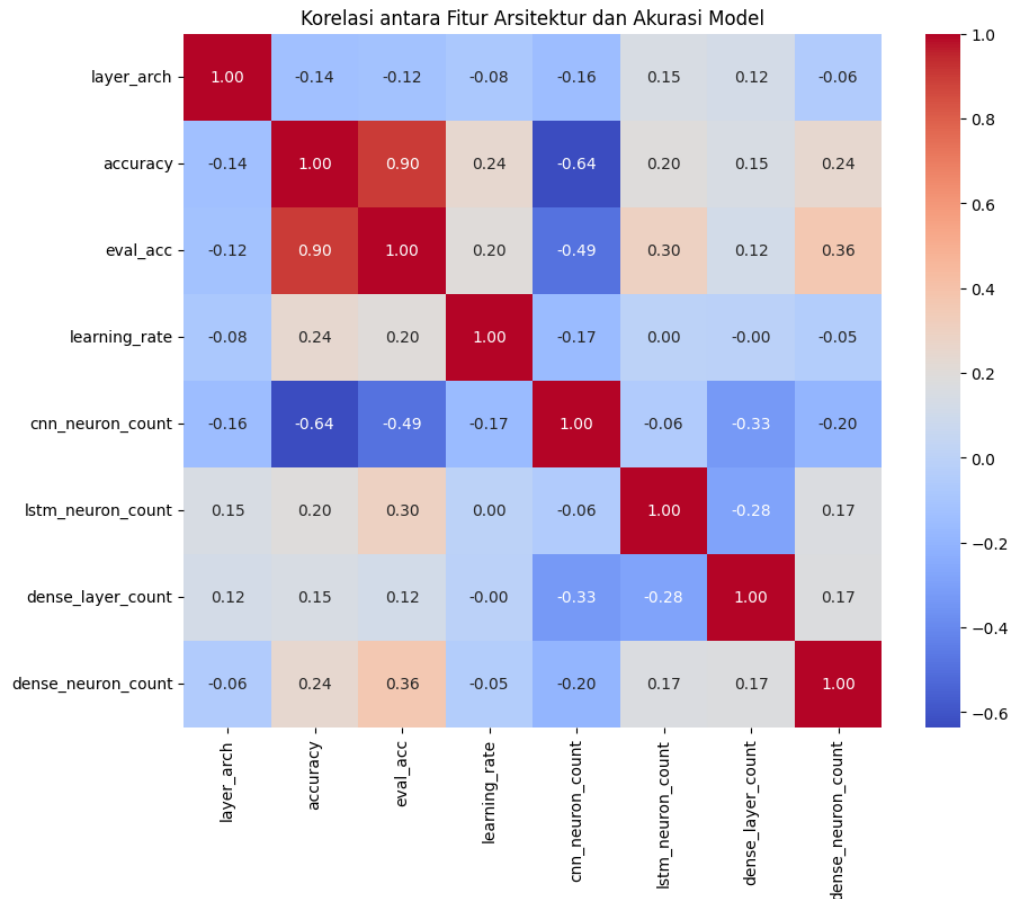
Gambar 4.2.6 Grafik Akurasi Arsitektur 2LSTM 1CNN



Gambar 4.2.7 Grafik Rata Rata Akurasi Learning Rate

Pada pengujian pertama, peneliti mengevaluasi setiap kemungkinan skema pengujian untuk menentukan model yang optimal sebelum melakukan pengujian menggunakan metode K-Folding. Berdasarkan analisis terhadap enam grafik yang disajikan, dapat disimpulkan bahwa arsitektur dengan empat lapisan LSTM menghasilkan akurasi tertinggi, mencapai 0.9048 , serta akurasi pada data baru sebesar 0.9850 . Learning rate yang secara konsisten menghasilkan akurasi tinggi

adalah $1-e3$ (0.001), dengan rata-rata akurasi 0.7363 dan rata-rata akurasi pada data baru 0.6967.



Gambar 4.2.8 Grafik Heatmap Korelasi Tanpa K-Fold

Dalam matriks korelasi diatas, aspek yang paling mencolok adalah nilai korelasi antara *cnn_neuron_count* (jumlah lapisan CNN beserta neuronnya) dengan *accuracy* (nilai akurasi pelatihan), yang tercatat sebesar -0.64 . Hal ini menandakan adanya hubungan korelasi negatif antara kedua variabel tersebut, yang mengimplikasikan bahwa peningkatan jumlah lapisan CNN berpotensi menurunkan nilai akurasi. Oleh karena itu, untuk mencapai akurasi pelatihan yang lebih tinggi, diperlukan konfigurasi dengan jumlah lapisan CNN yang lebih sedikit.

Dalam kontras dengan nilai *cnn_neuron_count*, nilai dari *lstm_neuron_count* dan *dense_neuron_count* menunjukkan kecenderungan positif. Meskipun tidak signifikan, hal ini mengindikasikan bahwa peningkatan jumlah lapisan LSTM dan dense dapat berkontribusi pada peningkatan akurasi.

4.2.2 Pengujian Dengan K-Fold

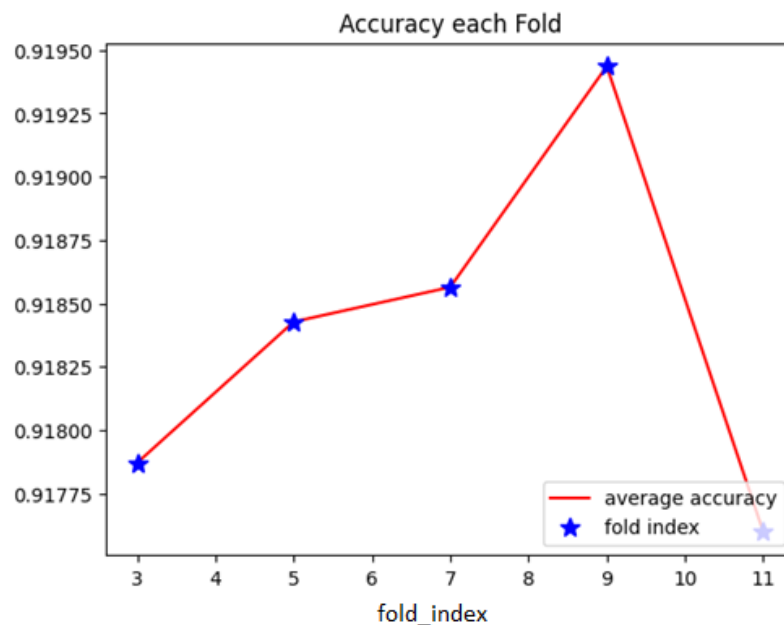
```
Best val_accuracy So Far: 0.9143134355545044
Total elapsed time: 00h 41m 28s
```

Gambar 4.2.9 Akurasi Data Baru dengan K-Fold

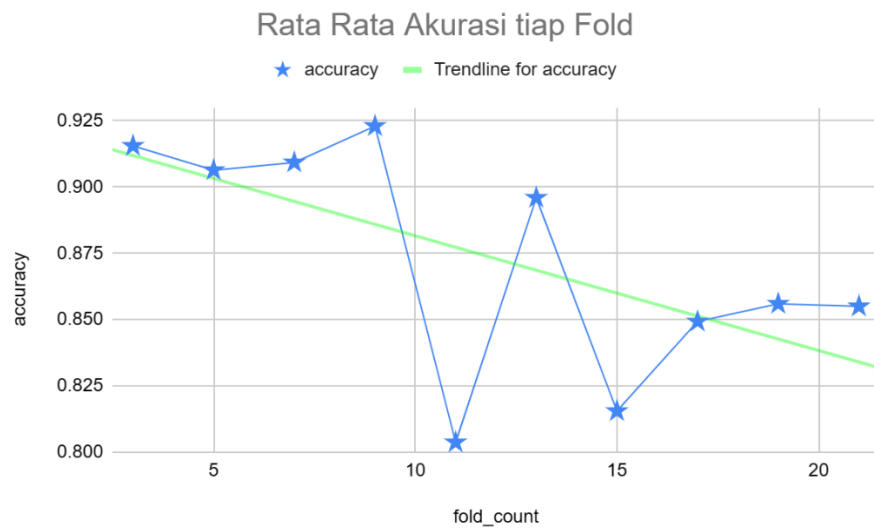
```
Fold ke 9 validation Loss: 0.207, Accuracy: 0.920,
epoch: 8, time_elapse: 5.378
```

```
-----
Average 9-fold accuracy: 0.9238836103015475
-----
```

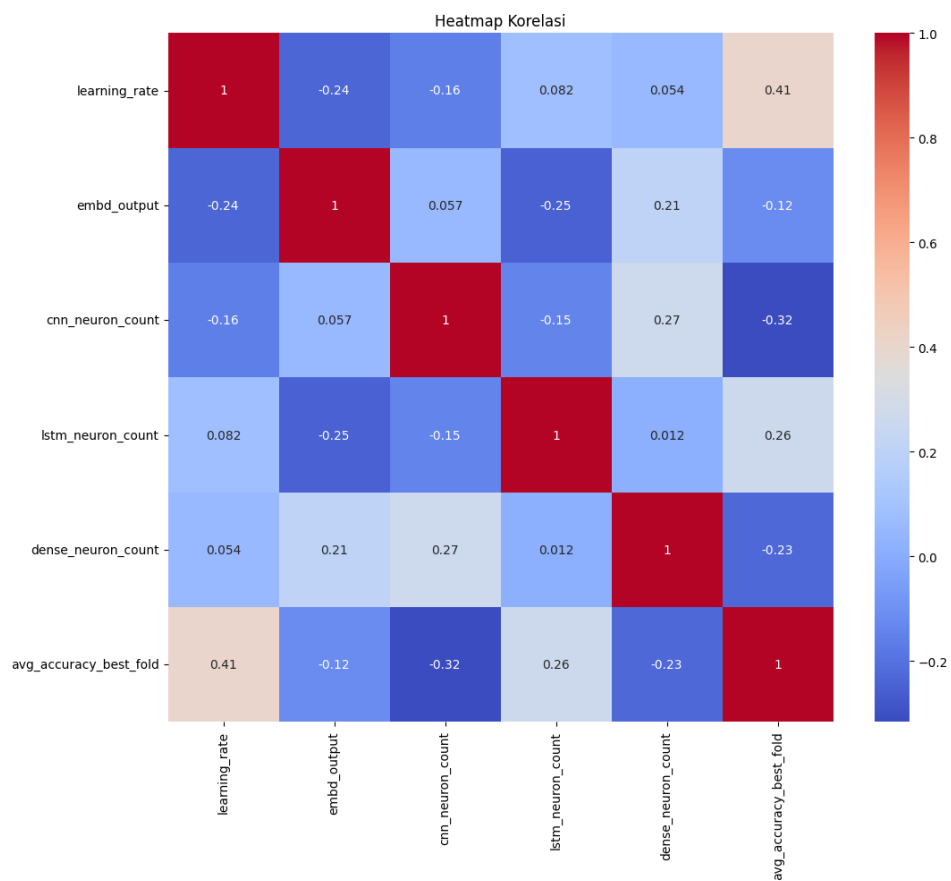
Gambar 4.2.10 Akurasi Data Latih dengan K-Fold



Gambar 4.2.11 Pengujian Pertama Crash



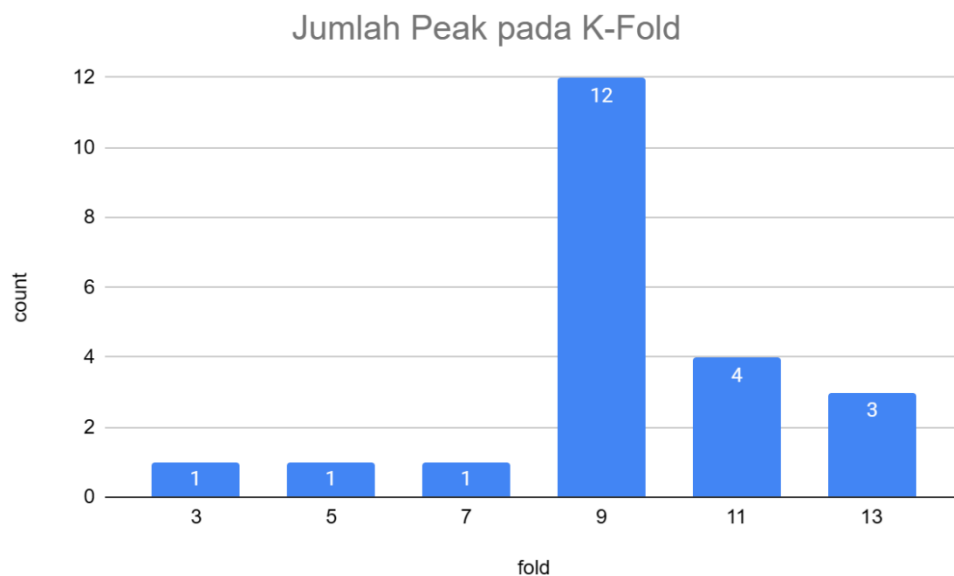
Gambar 4.2.12 Pengujian dengan Melakukan Fold Satu Persatu



Gambar 4.2.13 Grafik Heatmap Korelasi K-Fold

Dalam matriks korelasi di atas yang menggunakan validasi silang *K-Fold* sebagai penentu akurasi, terlihat adanya pola interaksi variabel yang serupa dengan matriks korelasi tanpa *K-Folding*; namun dengan nilai *learning_rate* yang lebih tinggi.

Penggunaan K-Fold ditujukan untuk mendapatkan stabilitas performa antara pengujian data baru dengan data latih, dan terbukti di Gambar 4.2.9 dan Gambar 4.2.10.



Gambar 4.2.14 Grafik Persebaran Puncak Akurasi Tertinggi

Pengujian pada Gambar 4.2.11 dilakukan dengan sekali menjalankan program dan mengalami crash di fold ke 13. Pada gambar tersebut dapat dilihat bahwa puncak akurasi tertinggi ada pada fold ke-9. Sedangkan pada Gambar 4.2.12, dimana program dijalankan sekali tiap foldnya, menunjukkan puncak tertinggi pada fold ke-9.

Akurasi tertinggi pada pengujian menggunakan K-Fold adalah **0.92990** dengan arsitektur:

- 64 Neuron Emmbeding layer
- 4-layer LSTM dengan neuron [128, 128, 128, 128]
- Satu layer dense dengan neuron 32
- learning rate 0.001 (1-e3)

Arsitektur ini berhasil dijalankan dengan waktu **119.45** menit pada fold dengan akurasi tertinggi di **11-fold**.

Kemudian, jika melihat dari waktu latih rendah dengan akurasi tinggi, didapatkan arsitektur:

- 128 Neuron Emmbeding layer
- 3-layer LSTM dengan neuron [128, 128, 128]
- Satu layer CNN dengan neuron 128
- Satu layer Dense dengan neuron 16
- learning rate 0.001 (1-e3)

Arsitektur ini berhasil dijalankan dengan waktu **30.86** menit pada fold dengan akurasi tertinggi **0.92302** di **9-fold**.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Dengan memanfaatkan sampel data dari Kaggle sebagai sumber data, Penelitian ini dilakukan sebagai respons terhadap maraknya pencurian akun YouTube akibat url phishing yang terjadi antara bulan Januari hingga Mei 2023. Data yang diambil dari Kaggle mencakup set data yang dimiliki oleh Manu Siddhartha pada tahun 2021, ditambah data white-list dari Phishing Database milik mitchellkrogza pada github. Dalam percobaan pertama, peneliti melakukan eksperimen tanpa menggunakan metode K-Fold dengan tujuan untuk menemukan arsitektur model terbaik. Pada percobaan kedua, peneliti menggunakan metode K-Fold dalam satu run dan satu run per setiap Fold. Hasil dari penelitian ini adalah:

1. Komposisi layer LSTM penuh dapat menghasilkan akurasi tertinggi tetapi dengan bayaran komputasi yang lebih lama. Sedikit layer CNN (4 layer total dengan 1CNN dan sisanya LSTM) akan mempercepat komputasi secara signifikan dengan bayaran akurasi yang sedikit lebih rendah. Selain dua komposisi diatas, akan menghasilkan akurasi yang rendah bahkan sangat rendah.
2. Hyper parameter yang paling berdampak pada akurasi klasifikasi adalah jumlah layer CNN, jika terlalu banyak jumlah layernya maka akan menurunkan akurasi secara drastis, kemudian learning rate, jika terlalu

lambat maka proses belajar dari model akan terhambat diangka yang rendah.

Berdasarkan temuan penelitian ini, peneliti menyimpulkan bahwa penggabungan arsitektur Convolutional Neural Network (CNN) dengan Long Short-Term Memory (LSTM) secara efektif dapat digunakan untuk kasus klasifikasi. Terutama, dalam konteks pengklasifikasian URL, pendekatan ini menunjukkan efisiensi waktu dan akurasi yang baik.

5.2. Saran

Berdasarkan penelitian yang sudah dilaksanakan oleh peneliti, didapatkan beberapa saran dari hasil penelitian ini antara lain:

1. Penelitian dapat dilakukan pada perangkat dengan spesifikasi yang lebih mumpuni.
2. Mencoba menggunakan transfer learning agar mendapatkan akurasi yang lebih tinggi.
3. Mencoba menggunakan model tokenizer yang lebih advance seperti LLM, LLAMA atau GPT-Tokenizer.

DAFTAR PUSTAKA

- [1] H. huan Wang, L. Yu, S. wei Tian, Y. fang Peng, and X. jun Pei, "Bidirectional LSTM Malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network," *Appl. Intell.*, vol. 49, no. 8, pp. 3016–3026, Aug. 2019, doi: 10.1007/s10489-019-01433-4.
- [2] W. Wei, Q. Ke, J. Nowak, M. Korytkowski, R. Scherer, and M. Woźniak, "Accurate and fast URL phishing detector: A convolutional neural network approach," *Comput. Netw.*, vol. 178, Sep. 2020, doi: 10.1016/j.comnet.2020.107275.
- [3] A. Crisan, G. Florea, L. Halasz, C. Lemnaru, and C. Oprisa, "Detecting Malicious URLs Based on Machine Learning Algorithms and Word Embeddings," in *Proceedings - 2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing, ICCP 2020*, Institute of Electrical and Electronics Engineers Inc., Sep. 2020, pp. 187–193. doi: 10.1109/ICCP51029.2020.9266139.
- [4] P. Wanda and H. J. Jie, "URLDeep: Continuous Prediction of Malicious URL with Dynamic Deep Learning in Social Networks," *Int. J. Netw. Secur.*, vol. 21, pp. 971–978, Nov. 2019, doi: 10.6633/IJNS.846.
- [5] Z. Wang, X. Ren, S. Li, B. Wang, J. Zhang, and T. Yang, "A Malicious URL Detection Model Based on Convolutional Neural Network," *Secur. Commun. Netw.*, vol. 2021, 2021, doi: 10.1155/2021/5518528.
- [6] D. Harvey M, D. Paul J, and R. Nieto T, *Internet and World Wide Web: How to Program*. Prentice Hall, 2001. [Online]. Available: www.theadmin.data.bg
- [7] N. Gupta, S. Mujumdar, P. Agarwal, A. Jain, and S. Mehta, "Learning Convolutional Neural Networks with Deep Part Embeddings," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, United Kingdom: IEEE, May 2019, pp. 2037–2041. doi: 10.1109/ICASSP.2019.8682192.
- [8] Giancarlo. Zacccone and Md. Rezaul. Karim, *Deep Learning with TensorFlow : Explore neural networks and build intelligent systems with Python, 2nd Edition*. Packt Publishing, 2018.
- [9] T. Oliver, *Machine Learning for Absolute Beginners: A Plain English Introduction*. Scatterplot Press, 2020.
- [10] G. Hirst and Y. Goldberg, *Neural Network Methods for Natural Language Processing*. Morgan & Claypool, 2017. [Online]. Available: <http://store.morganclaypool.com>
- [11] Alexandra George, *Python Text Mining: Perform Text Processing, Word Embedding, Text Classification and Machine Translation*. BPB Publications, 2022.
- [12] G. S. Mahara and S. Gangele, "Fake news detection: A RNN-LSTM, Bi-LSTM based deep learning approach," in *2022 IEEE 1st International Conference on Data, Decision and Systems (ICDDS)*, Dec. 2022, pp. 01–06. doi: 10.1109/ICDDS56399.2022.10037403.

- [13] H. Hafid, “Penerapan K-Fold Cross Validation untuk Menganalisis Kinerja Algoritma K-Nearest Neighbor pada Data Kasus Covid-19 di Indonesia,” 2023.
- [14] G. Aurélien, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O’Reilly Media, 2019.

LAMPIRAN

Lampiran 1 Gambar Contoh Hasil Layer LSTM

```
<tf.Tensor: shape=(1, 4), dtype=float32, numpy=
array([[ -0.01316059,  0.0015068 , -0.00633642, -0.0036962 ]],
      dtype=float32)>
```

Lampiran 2 Seluruh Data Pengujian tanpa K-Fold

layer_arch	accuracy	eval_acc	learning_rate	cnn_neuron_count	lstm_neuron_count	dense_layer_count	dense_neuron_count
CNN(0), LSTM(4)	0.9048	0.9850	0.001	N/A	128, 128, 64, 32	3	256, 32, 512
CNN(1), LSTM(3)	0.9005	0.9488	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8965	0.9065	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8937	0.8333	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8736	0.5258	0.0001	256	256, 128, 128	2	128, 64
CNN(2), LSTM(2)	0.8706	0.7208	0.001	256, 128	128, 128	2	128, 64
CNN(0), LSTM(4)	0.8705	0.7583	0.001	N/A	256, 128, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8695	0.7437	0.001	256	128, 128, 128	1	64
CNN(1), LSTM(2)	0.8647	0.7158	0.001	256	128, 128	3	128, 64, 32
CNN(1), LSTM(2)	0.8602	0.7054	0.001	256	128, 129	3	128, 64, 32
CNN(2), LSTM(2)	0.8600	0.7050	0.001	256, 128	128, 128	3	128, 64, 32
CNN(2), LSTM(2)	0.8549	0.7417	0.001	256, 128	128, 128	3	128, 64, 32
CNN(1), LSTM(2)	0.8513	0.7479	0.001	256	128, 128	3	128, 64, 32
CNN(1), LSTM(3)	0.8466	0.9300	0.001	64	128, 32, 32	2	512, 32
CNN(2), LSTM(2)	0.8459	0.6800	0.001	256, 128	128, 128	3	128, 64, 32
CNN(1), LSTM(3)	0.8269	0.7267	0.008	256	256, 128, 128	2	128, 64
CNN(0), LSTM(4)	0.8071	0.8675	0.0001	N/A	64, 512, 512, 32	2	512, 64
CNN(1), LSTM(3)	0.8070	0.8650	0.0001	256	128, 256, 64	2	128, 32
CNN(3), LSTM(1)	0.7473	0.7150	0.001	256, 128, 128	128	2	128, 64
CNN(2), LSTM(2)	0.7440	0.7050	0.001	256, 128	128, 128	2	128, 64
CNN(2), LSTM(2)	0.7422	0.7013	0.001	256, 128	128, 128	4	128, 64, 32, 16
CNN(2), LSTM(2)	0.7405	0.7405	0.000001	256, 128	128, 128	3	128, 64, 32
CNN(2), LSTM(2)	0.7183	0.8800	0.001	512, 128	512, 256	2	32, 32
CNN(2), LSTM(2)	0.6924	0.8000	0.0001	128, 128	512, 128	3	64, 512, 128
CNN(1), LSTM(3)	0.5927	0.5575	0.00001	512	64, 256, 512	1	128
CNN(0), LSTM(4)	0.5515	0.4300	0.00001	N/A	512, 256, 512, 128	3	128, 64, 256

CNN(2), LSTM(2)	0.5407	0.4600	0.00001	256, 512	128, 32	1	32
CNN(1), LSTM(2)	0.4966	0.4979	0.001	256	128, 128	3	128, 64, 32
CNN(3), LSTM(1)	0.3991	0.4750	0.001	32, 128, 256	32	2	32, 32
CNN(3), LSTM(1)	0.3929	0.4300	0.0001	64, 256, 128	256	3	128, 128, 512
CNN(3), LSTM(1)	0.3722	0.4075	0.00001	256, 512, 128	32	1	128
CNN(4), LSTM(0)	0.3058	0.2994	0.001	256, 128, 64, 32	N/A	2	128, 64
CNN(4), LSTM(0)	0.3030	0.2954	0.0001	256, 128, 64, 32	N/A	2	128, 64
CNN(4), LSTM(0)	0.3000	0.3150	0.001	256, 128, 128, 128	N/A	3	128, 64
CNN(4), LSTM(0)	0.2943	0.2925	0.0001	32, 32, 128, 64	N/A	3	32, 256, 256
CNN(4), LSTM(0)	0.2926	0.3150	0.001	64, 256, 32, 128	N/A	1	128
CNN(4), LSTM(0)	0.2882	0.2775	0.00001	64, 128, 256, 32	N/A	2	128, 256

Lampiran 3 Seluruh Data Pengujian Menggunakan K-Fold

fold_coun t	learn ing_r ate	embd _out put	layer _arc h	cn n_ ne u ro n_ c o u n t	lstm _ne u ro n_ c o u n t	dense _l a y e r_ c o u n t	dense _n e u ro n_ c o u n t	avg_ac c u r a c y_ b e s t_ f o l d	time_el a p s_ b e s t_ f o l d	avg_ep o c h_ b e s t_ f o l d	K- Fold_ meth o d	best_f o l d
13	0.001	64	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	1	32	0.92990	119.45	6	Stratifi ed	11
11	0.001	256	CNN(0), LSTM(4)	N/A	128, 128, 256, 256	2	16, 64	0.92827	65.18	6	Stratifi ed	9
13	0.001	128	CNN(0), LSTM(4)	N/A	256, 128, 256, 128	2	64, 16	0.92718	58.00	7	Non- Stratifi ed	9
13	0.001	64	CNN(0), LSTM(4)	N/A	128, 256, 128, 256	1	128	0.92393	83.76	7	Stratifi ed	13
9	0.001	64	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	2	128, 64	0.92388	42.04	7	Stratifi ed	9
9	0.001	128	CNN(1), LSTM(3)	128	128, 128, 128	1	16	0.92302	30.86	6	Stratifi ed	9
11	0.001	64	CNN(1), LSTM(3)	256	128, 256, 128	2	64, 32	0.92108	48.15	6	Stratifi ed	9
11	0.001	128	CNN(1), LSTM(3)	128	256, 128, 128	1	32	0.91948	43.60	6	Stratifi ed	11
13	0.001	32	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	2	64, 128	0.91928	59.34	7	Stratifi ed	13
3	0.001	32	CNN(0), LSTM(4)	N/A	128, 256, 128, 128	2	32, 64	0.91800	20.80	7	Stratifi ed	3
13	0.001	64	CNN(1), LSTM(3)	128	128, 128, 128	2	16, 32	0.91760	56.40	7	Stratifi ed	11
11	0.001	16	CNN(0), LSTM(4)	N/A	256, 256, 128, 128	2	64, 128	0.91724	109.83	7	Stratifi ed	9
9	0.001	64	CNN(1), LSTM(3)	256	128, 256, 256	1	16	0.91700	38.56	6	Stratifi ed	9
9	0.001	16	CNN(1), LSTM(3)	128	128, 64, 64	2	32, 32	0.91501	45.70	7	Non- Stratifi ed	9

11	0.001	64	CNN(1), LSTM(3)	128	128, 256, 256	1	64	0.91500	39.37	7	Stratified	9
9	0.001	16	CNN(1), LSTM(3)	128	128, 128, 128	1	32	0.91115	23.51	7	Stratified	7
15	0.001	16	CNN(1), LSTM(3)	128	128, 256, 256	2	128, 128	0.90714	58.27	7	Stratified	13
13	0.0001	128	CNN(1), LSTM(3)	256	128, 256, 128	2	64, 16	0.89967	70.48	7	Stratified	11
5	0.0001	256	CNN(1), LSTM(3)	128	128, 128, 128	2	32, 64	0.89115	17.37	7	Non-Stratified	5
11	0.0001	16	CNN(1), LSTM(3)	128	128, 256, 128	3	16, 16, 128	0.89054	58.42	8	Stratified	9
9	0.0001	64	CNN(0), LSTM(4)	N/A	128, 256, 256, 256	2	128, 64	0.88647	80.29	7	Non-Stratified	9
5	0.001	128	CNN(2), LSTM(2)	128, 64	128, 128	2	64, 32	0.82880	14.07	8	Non-Stratified	9

Lampiran 4 Arsitektur Model Terbaik Pengujian K-Fold

Model: "sequential"

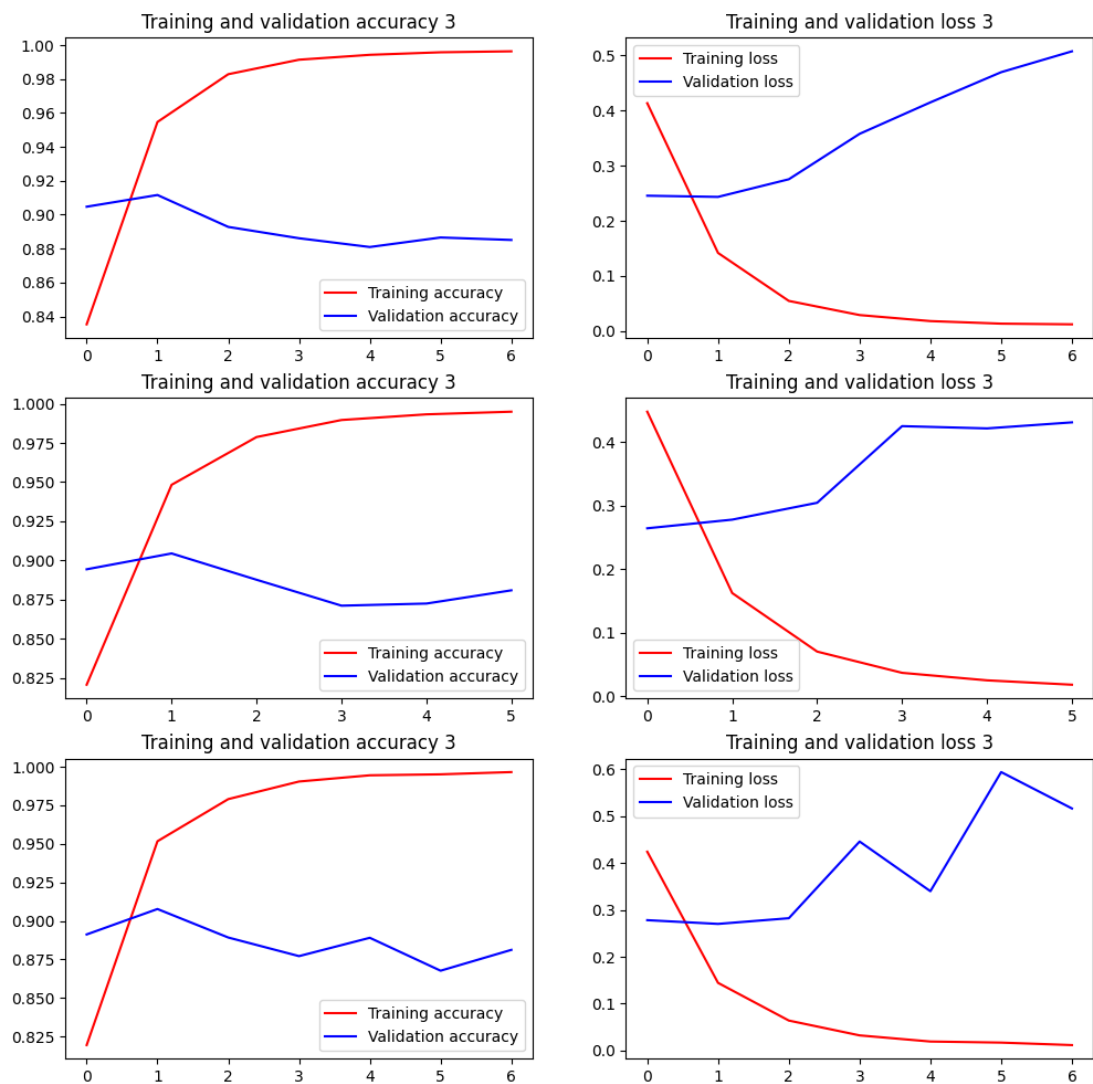
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 157, 64)	7441216
lstm (LSTM)	(None, 157, 128)	98816
lstm_1 (LSTM)	(None, 157, 128)	131584
lstm_2 (LSTM)	(None, 157, 128)	131584
lstm_3 (LSTM)	(None, 128)	131584
dense (Dense)	(None, 32)	4128
dense_1 (Dense)	(None, 4)	132

```

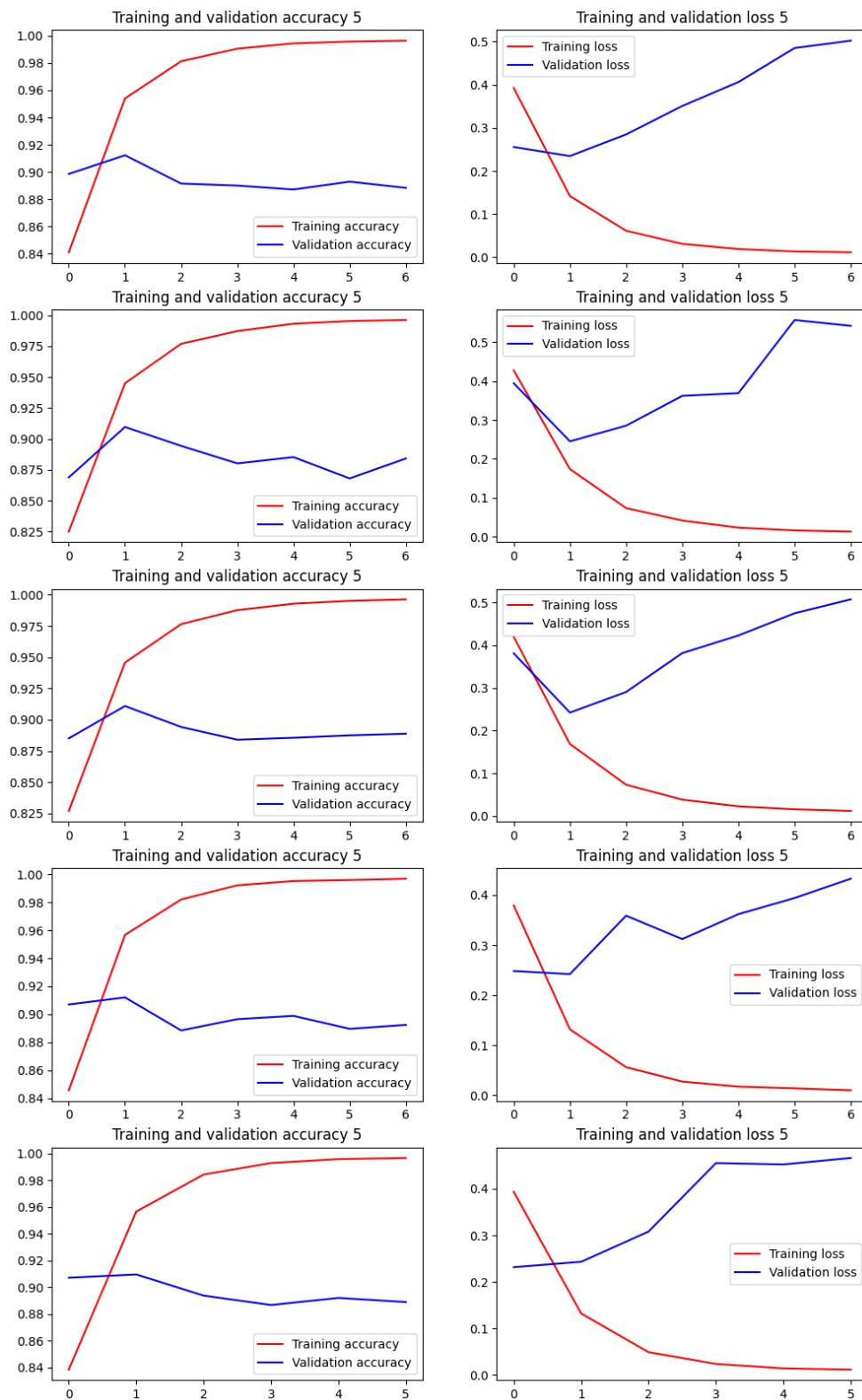
=====
Total params: 7939044 (30.29 MB)
Trainable params: 7939044 (30.29 MB)
Non-trainable params: 0 (0.00 Byte)
=====
Learning Rate: 0.001

```

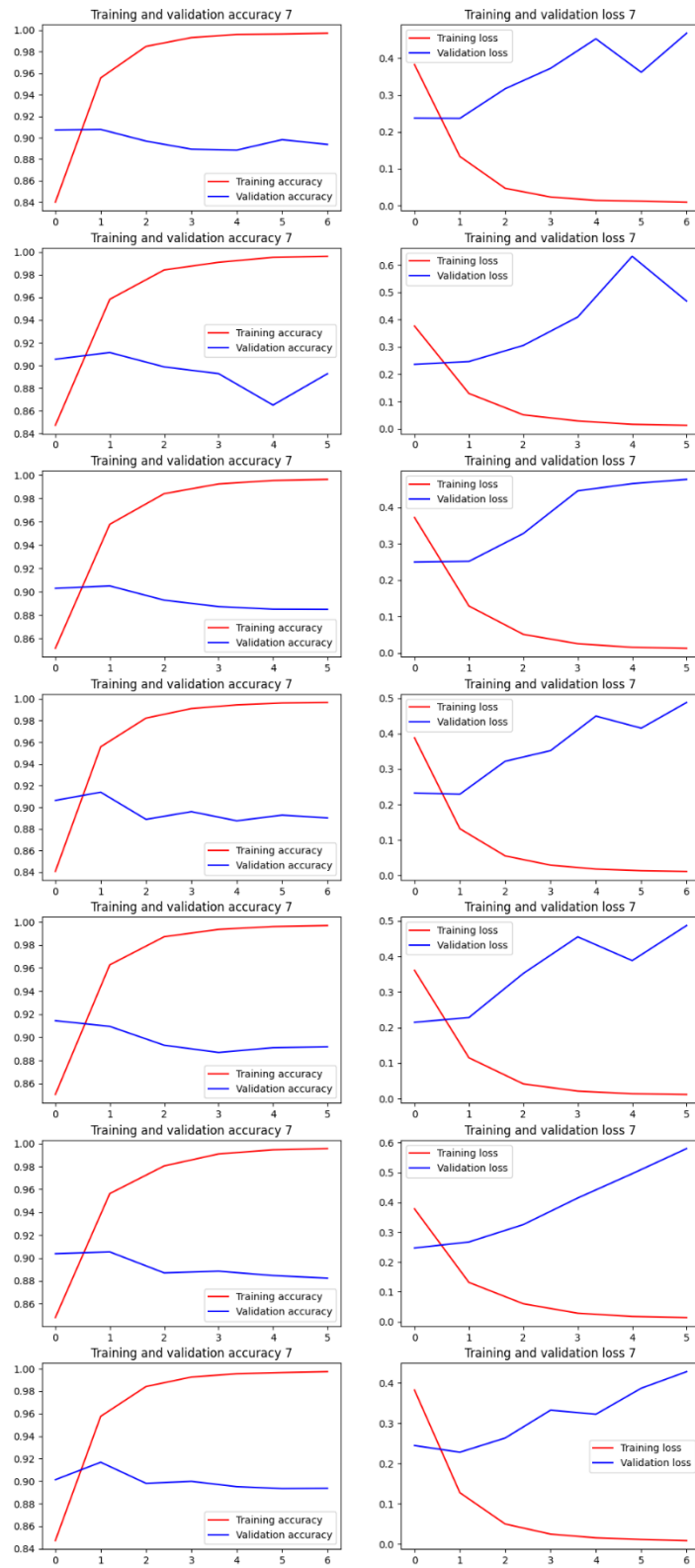
Lampiran 5 Grafik akurasi dan loss tiap iterasi 3-fold



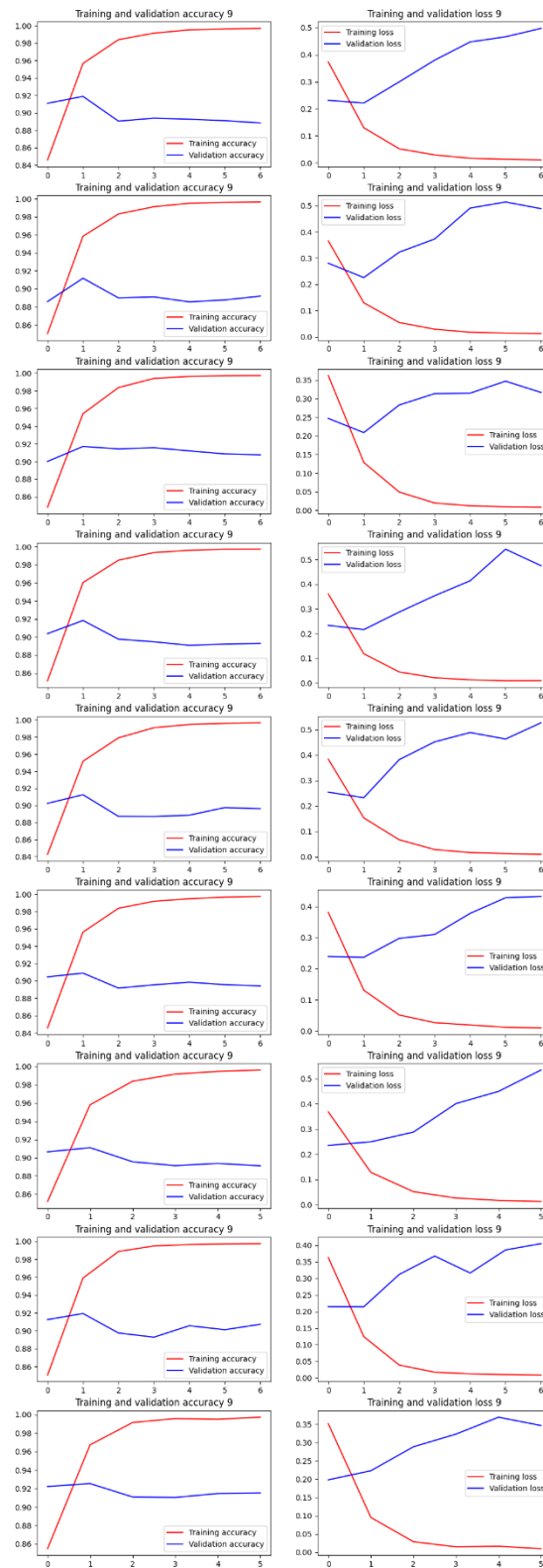
Lampiran 6 Grafik akurasi dan loss tiap iterasi 5-fold



Lampiran 7 Grafik akurasi dan loss tiap iterasi 7-fold



Lampiran 8 Grafik akurasi dan loss tiap iterasi 9-fold



Lampiran 9 Error yang didapat tiap run

```
Node: 'Adam/StatefulPartitionedCall'
Failed to load in-memory CUBIN (compiled for a different GPU?): CUDA_ERROR_OUT_OF_MEMORY:
out of memory
      [{{(node Adam/StatefulPartitionedCall)}}] [Op:__inference_train_function_3392948]
```

```
-----
Training with 11-fold cross-validation
-----
```

```
Fold ke 1 validation_loss: 0.216, accuracy: 0.904, epoch: 6, time_elapse: 4.541 mnt
Fold ke 2 validation_loss: 0.234, accuracy: 0.910, epoch: 7, time_elapse: 5.223 mnt
Fold ke 3 validation_loss: 0.223, accuracy: 0.911, epoch: 7, time_elapse: 5.254 mnt
Fold ke 4 validation_loss: 0.243, accuracy: 0.900, epoch: 6, time_elapse: 4.518 mnt
Fold ke 5 validation_loss: 0.234, accuracy: 0.910, epoch: 6, time_elapse: 4.531 mnt
Fold ke 6 validation_loss: 0.223, accuracy: 0.908, epoch: 6, time_elapse: 4.557 mnt
Fold ke 7 validation_loss: 0.210, accuracy: 0.916, epoch: 7, time_elapse: 5.212 mnt
Fold ke 8 validation_loss: 0.228, accuracy: 0.908, epoch: 6, time_elapse: 4.553 mnt
Fold ke 9 validation_loss: 0.219, accuracy: 0.909, epoch: 6, time_elapse: 4.538 mnt
Fold ke 10 validation_loss: 0.251, accuracy: 0.910, epoch: 7, time_elapse: 5.199 mnt
An error occurred: Graph execution error
```